



Learning/Training Documents

SIEMENS AUTOMATION COOPERATES WITH EDUCATION (SCE)

TIA Portal Module 031-200 | From Version V14
Basics of FB Programming with SIMATIC S7-1200

[siemens.com/sce](https://www.siemens.com/sce)

SIEMENS

Matching SCE Trainer Packages for this Learning/Training Document

Please note that these trainer packages are replaced with successor packages when necessary.

An overview of the currently available SCE packages is provided at: siemens.com/sce/tp

Controllers

- **Basic controller SIMATIC S7-1200, CPU 1215C; AC/DC/RELAIS**

Order No.: 6ES7215-1BG40-4AB1

- **Basic controller SIMATIC S7-1200, CPU 1215C; DC/DC/DC**

Order No.: 6ES7215-1AG40-4AB1

SIMATIC STEP 7 Software for Training

Form of delivery: DVD

- **Upgrade SIMATIC TIA Portal Engineering Software STEP 7 Basic for S7-1200 as of V10.5**

Order No.: 6ES7822-0AA24-4YE5

Form of delivery: OSD – Online Software Delivery (OSD) = Software downloads via email address of the end user, e.g. educator/installer in educational institutions, not of the distributor.

- **Upgrade SIMATIC TIA Portal Engineering Software STEP 7 Basic for S7-1200 as of V10.5**

Order No.: 6ES7822-0AE24-4YE5

Training courses

For Siemens SCE training courses in your region, contact your regional SCE representative:

siemens.com/sce/contact

Additional information regarding SCE

siemens.com/sce

Information regarding use

This SCE Learning/Training Document for the integrated automation solution Totally Integrated Automation (TIA) was prepared for the program "Siemens Automation Cooperates with Education (SCE)" specifically for training purposes for public educational facilities and R&D institutions as well as company training centers. Siemens AG assumes no responsibility for the content.

This document is to be used only for initial training on Siemens products/systems. This means it can be copied in whole or in part and given to those being trained for use within the scope of their training. Disseminating or copying this document and sharing its content is permitted within public training and advanced training schools for training purposes.

Exceptions require written consent from Siemens AG. Send all related requests to

scsupportfinder.i-ia@siemens.com.

Offenders will be liable for damages. All rights reserved, including those relating to translation, and particularly those rights created as a result of a patent being granted, or utility model being registered.

Use for industrial customer courses is expressly prohibited. We do not consent to commercial use of the documents.

We would like to thank Michael Dziallas Engineering and all those involved for their support in creating this SCE learning/training document.

Table of contents

1	Goal	5
2	Prerequisite	5
3	Required hardware and software	6
4	Theory	7
4.1	Operating system and application program	7
4.2	Organization blocks	8
4.3	Process image and cyclic program processing	9
4.4	Functions	11
4.5	Function blocks and instance data blocks	12
4.6	Global data blocks	13
4.7	Library-compatible logic blocks	14
4.8	Programming languages	15
5	Task	16
6	Planning	16
6.1	EMERGENCY STOP	16
6.2	Automatic mode - Conveyor motor	16
6.3	Technology diagram	17
6.4	Reference list	18
7	Structured step-by-step instructions	19
7.1	Retrieve an existing project	19
7.2	Create a new tag table	20
7.3	Create new tags within a tag table	22
7.4	Import "Tag_table_sorting_station"	23
7.5	Create function block FB1 "MOTOR_AUTO" for the conveyor motor in automatic mode	27
7.6	Define the interface of FB1 "MOTOR_AUTO"	29
7.7	Program FB1: MOTOR_AUTO	32
7.8	Program the organization block OB1 – Control conveyor tracking forwards in automatic mode	40
7.9	Result in the LAD (Ladder Logic) programming language	45
7.10	Save and compile the program	46
7.11	Download the program	47
7.12	Monitor program blocks	48
7.13	Archive the project	51
7.14	Checklist	52
8	Exercise	53
8.1	Task – Exercise	53
8.2	Technology diagram	53
8.3	Reference list	54
8.4	Planning	54
8.5	Checklist – Exercise	55
9	Additional information	56

Basics of FB Programming

1 Goal

In this chapter, you will get to know the basic elements of a control program – the **organization blocks (OBs)**, **functions (FCs)**, **function blocks (FBs)** and **data blocks (DBs)**. In addition, you will be introduced to **library-compatible** function and function block programming. You will get to know the **Function Block Diagram (FBD)** programming language and use it to program a function block (FB1) and an organization block (OB1).

The SIMATIC S7 controllers listed in Chapter 3 can be used.

2 Prerequisite

This chapter builds on the hardware configuration of SIMATIC S7 CPU1214C. However, other hardware configurations that has digital input and output board be used. For this chapter, you can use the following project, for example:

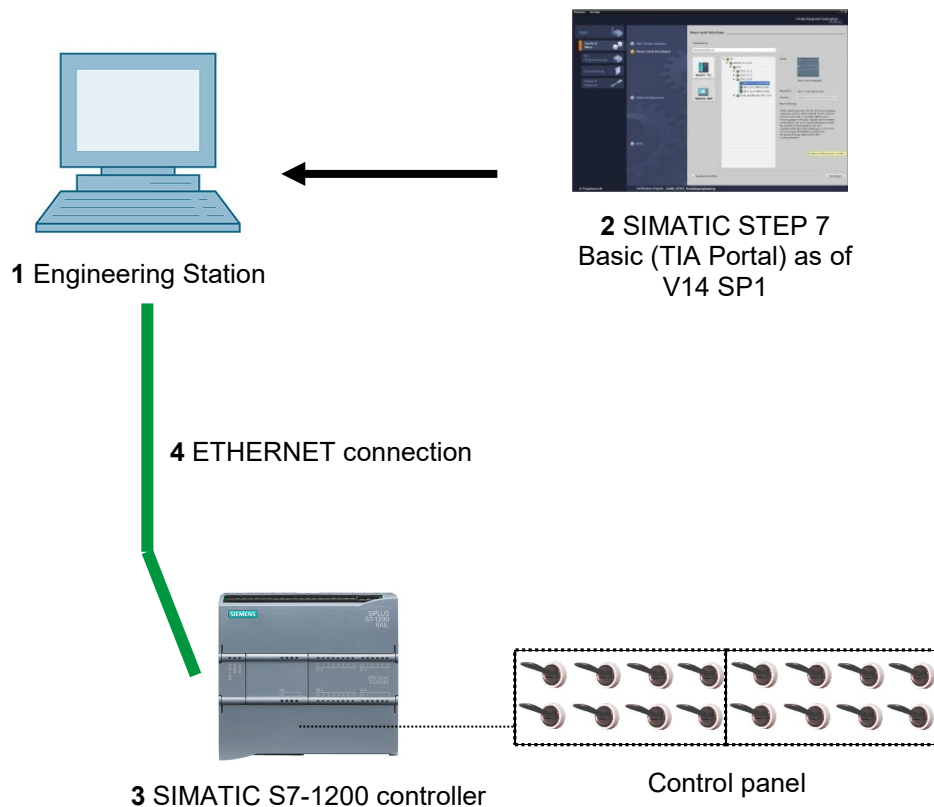
SCE_EN_011_101_Hardware_Configuration_CPU1214C.zap14

3 Required hardware and software

- 1 Engineering station: requirements include hardware and operating system (for additional information, see Readme on the TIA Portal Installation DVDs)
- 2 SIMATIC STEP 7 Basic software in TIA Portal – as of V14 SP1
- 3 SIMATIC S7-1200 controller, e.g. CPU 1214C DC/DC/DC with ANALOG OUTPUT SB1232 signal board, 1 AO – Firmware as of V4.2.1

Note: The digital inputs should be fed out to a control panel.

- 4 Ethernet connection between engineering station and controller



4 Theory

4.1 Operating system and application program

Every controller (CPU) contains an **operating system**, which organizes all functions and sequences of the CPU that are not associated with a specific control task.

The tasks of the operating system include the following:

- Performing a warm restart
- Updating the process image of the inputs and output
- Cyclically calling the user program
- Detecting interrupts and calling interrupt OBs
- Detecting and handling errors
- Managing memory areas

The operating system is an integral component of the CPU and comes pre-installed.

The **user program** contains all functions that are necessary for executing your specific automation task. The tasks of the user program include the following:

- Checking the basic requirements for a warm restart using startup OBs
- Processing of process data, i.e. activation of output signals as a function of the input signal states
- Reaction to interrupts and interrupt inputs
- Error handling during normal program execution

4.2 Organization blocks

Organization blocks (OBs) form the interface between the operating system of the controller (CPU) and the application program. They are called from the operating system and control the following operations:

- Cyclic program processing (e.g. OB1)
- Startup characteristics of the controller
- Interrupt-driven program processing
- Error handling

A project must have, at a minimum, **an organization block for cyclic program processing**. An OB is called by a **start event** as shown in Figure 1. In addition, the individual OBs have defined priorities so that, for example, an OB82 for error handling can interrupt the cyclic OB1.

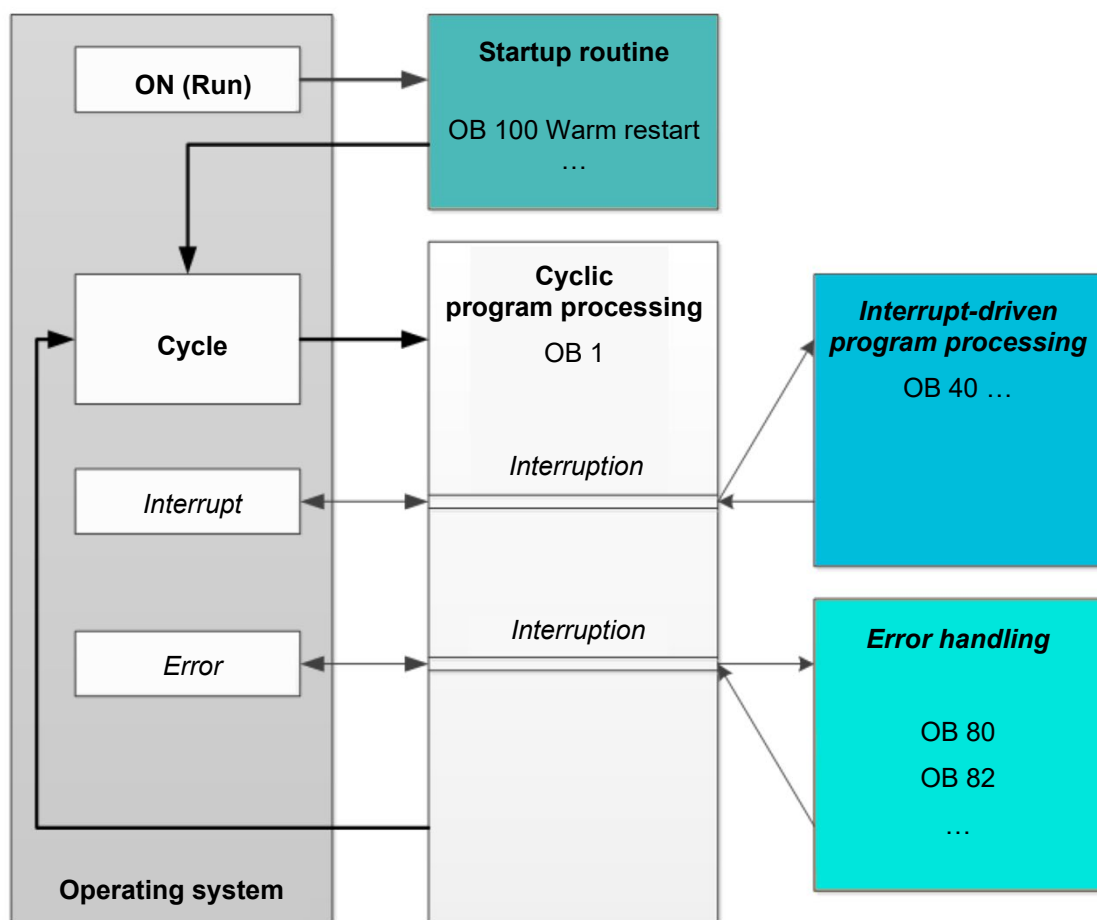


Figure 1: Start events in the operating system and OB calls

When a start event occurs, the following reactions are possible:

- If an OB has been assigned to the event, this event triggers the execution of the assigned OB. If the priority of the assigned OB is greater than the priority of the OB that is currently being executed, it is executed immediately (interrupt). If not, the assigned OB waits until the higher-priority OB has been completely executed
- If you have not assigned an OB to the event, the default system reaction is performed.

Table 1 shows examples for various start events for a SIMATIC S7-1200. Also shown are the possible OB number(s) and the default system reactions that occur when the respective organization block (OB) is not present in the controller.

Start event	Possible OB numbers	Default system reaction
Startup	100, ≥ 123	Ignore
Cyclic program	1, ≥ 123	Ignore
Time-of-day interrupt	10 to 11	-
Update interrupt	56	Ignore
Scan cycle monitoring time exceeded once	80	Ignore
Scan cycle monitoring time exceeded twice	80	STOP
Diagnostic interrupt	82	Ignore

Table 1: OB numbers for various start events

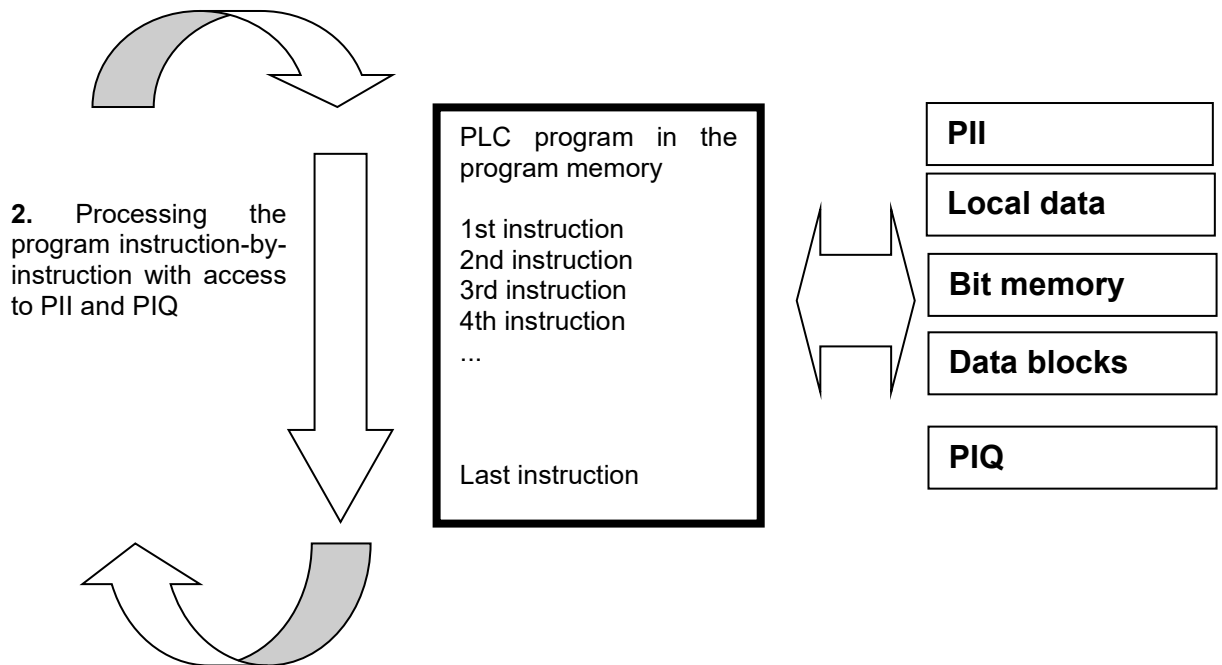
4.3 Process image and cyclic program processing

When the cyclic user program addresses the inputs (I) and outputs (O), it does not query the signal states directly from the input/output modules. Instead, it accesses a memory area of the CPU. This memory area contains an image of the signal states and is called the **process image**.

The cyclic program processing sequence is as follows:

1. At the start of the cyclic program, a query is sent to determine whether or not the individual inputs are energized. This status of the inputs is stored in the **process image of the inputs (PII)**. In doing so, the information 1 or "High" is stored for energized inputs and the information 0 or "Low" for de-energized inputs.
2. The CPU then executes the program stored in the cyclic organization block. For the required input information, the CPU accesses the previously read **process image of the inputs (PII)** and the results of logic operation (RLOs) are written to a so-called **process image of the outputs (PIQ)**.
3. At the end of the cycle, the **process image of the outputs (PIQ)** is transferred as the signal state to the output modules and these are energized or de-energized. The sequence then continues again with Item 1.

1. Save status of inputs in PII.



3. Transfer status from the PIQ to the outputs.

Figure 2: Cyclic program processing

Note:

- The time the CPU needs for this sequence is called *cycle time*. This depends, in turn, on the number and type of instructions and the processor performance of the controller.

4.4 Functions

Functions (FCs) are logic blocks without memory. They **have no data memory** in which values of block parameters can be stored. Therefore, all interface parameters must be connected when a function is called. To store data permanently, global data blocks must be created beforehand.

A function contains a program that is executed whenever the function is called from another logic block.

Functions can be used, for example, for the following purposes:

- Math functions – that return a result dependent on input values.
- Technological functions – such as individual controls with binary logic operations.

A function can also be called several times at different points within a program.

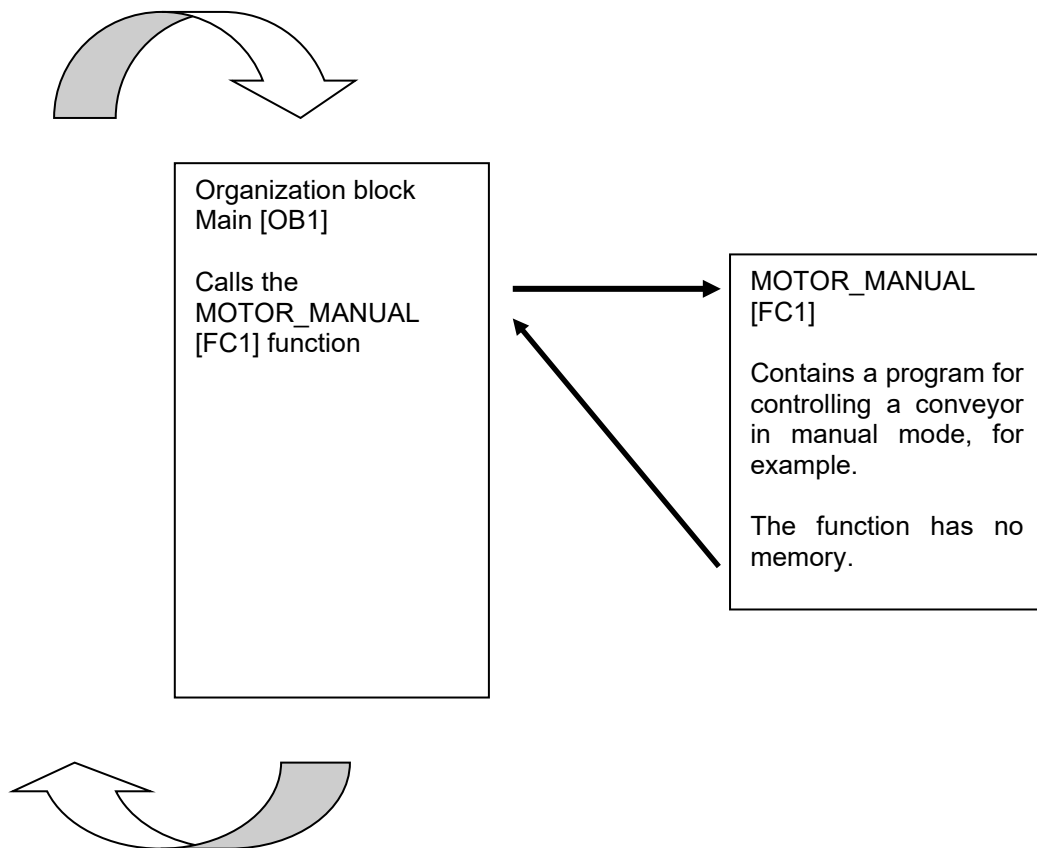


Figure 3: Function with call from organization block Main [OB1]

4.5 Function blocks and instance data blocks

Function blocks are logic blocks that store their input, output and in-out tags as well as static tags permanently in instance data blocks, so that they **are available after the block has been executed**. For this reason, they are also referred to as blocks with "memory".

Function blocks can also operate with temporary tags. Temporary tags are not stored in the instance DB, however. Instead, they are only available for one cycle.

Function blocks are used for tasks that cannot be implemented with functions:

- Whenever timers and counters are required in the blocks, or
- When information must be stored in the program, such as preselection of the operating mode with a button.

Function blocks are always executed when called from another logic block. A function block can also be called several times at different points within a program. This facilitates the programming of frequently recurring complex functions.

A call of a function block is referred to as an instance. Each instance of a function block is assigned a memory area that contains the data that the function block uses. This memory is made available by data blocks created automatically by the software.

It is also possible to provide memory for multiple instances in one data block in the form of a **multi-instance**. The maximum size of instance data blocks varies depending on the CPU. The tags declared in the function block determine the structure of the instance data block.

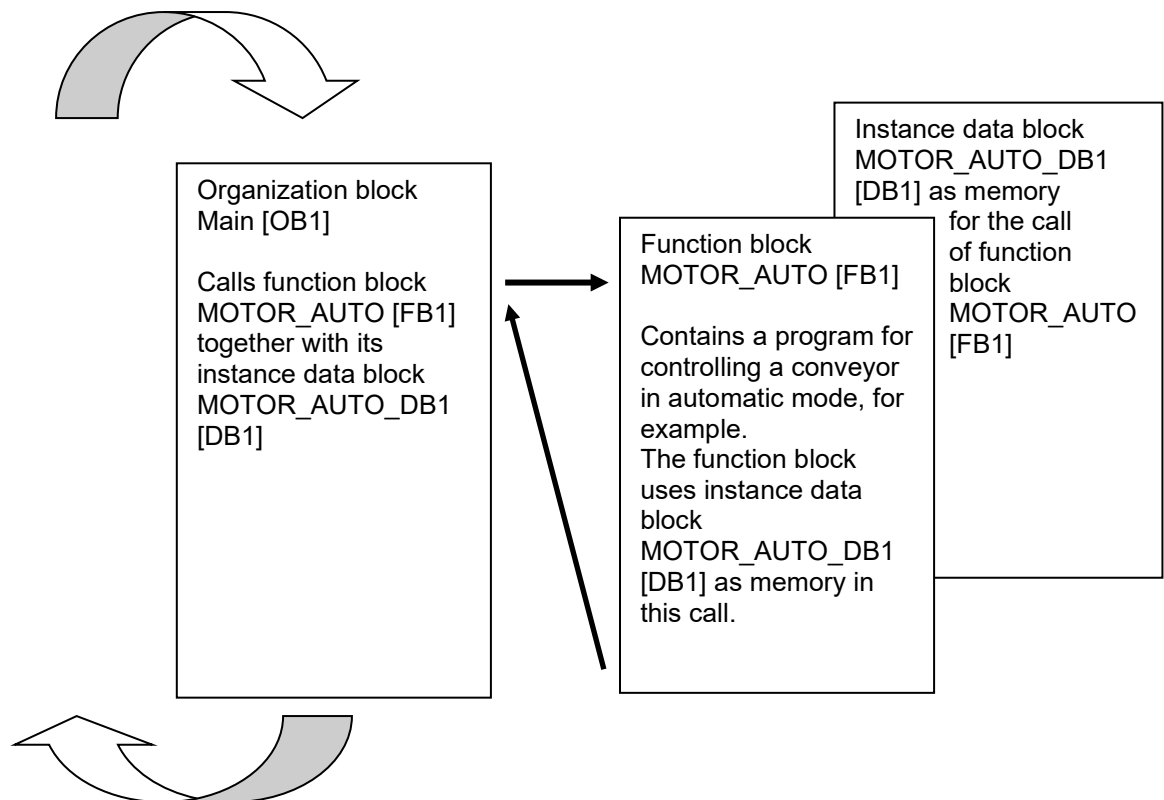


Figure 4: Function block and instance with call from organization block Main [OB1]

4.6 Global data blocks

In contrast to logic blocks, data blocks contain no instructions. Rather, they serve as memory for user data.

Data blocks thus contain variable data that is used by the user program. You can define the structure of global data blocks as required.

Global data blocks store data that can be used **by all other blocks** (see Figure 5). Only the associated function block should access instance data blocks. The maximum size of data blocks varies depending on the CPU.

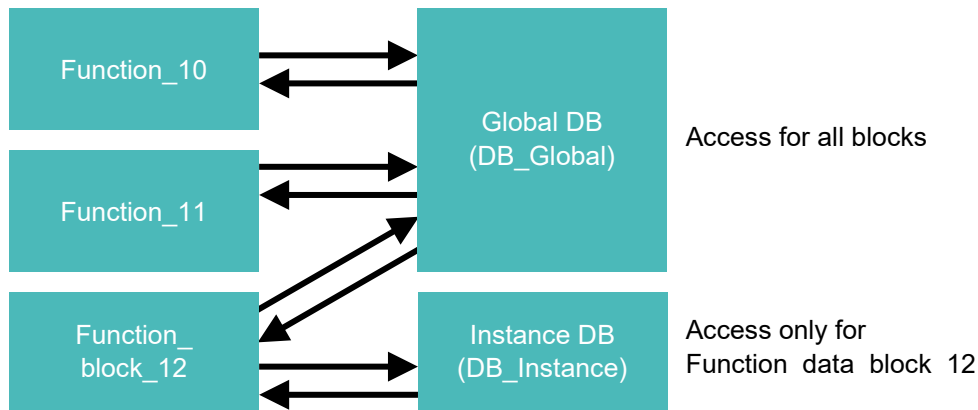


Figure 5: Difference between global DB and instance DB.

Application examples for **global data blocks** are:

- Saving of information about a storage system. "Which product is located where?"
- Saving of recipes for particular products.

4.7 Library-compatible logic blocks

A user program can be created with linear or structured programming. **Linear programming** writes the entire user program in the cycle OB, but is only suitable for very simple programs for which other less expensive control systems, such as LOGO!, can now be used.

For more complex programs, **structured programming** is always recommended. Here, the overall automation task can be broken down into small sub-tasks in order to implement a solution for them in functions and function blocks.

In this case, library-compatible logic blocks should be created preferentially. This means that the input and output parameters of a function or function block are defined generally and only supplied with the current global tags (inputs/outputs) when the block is used.

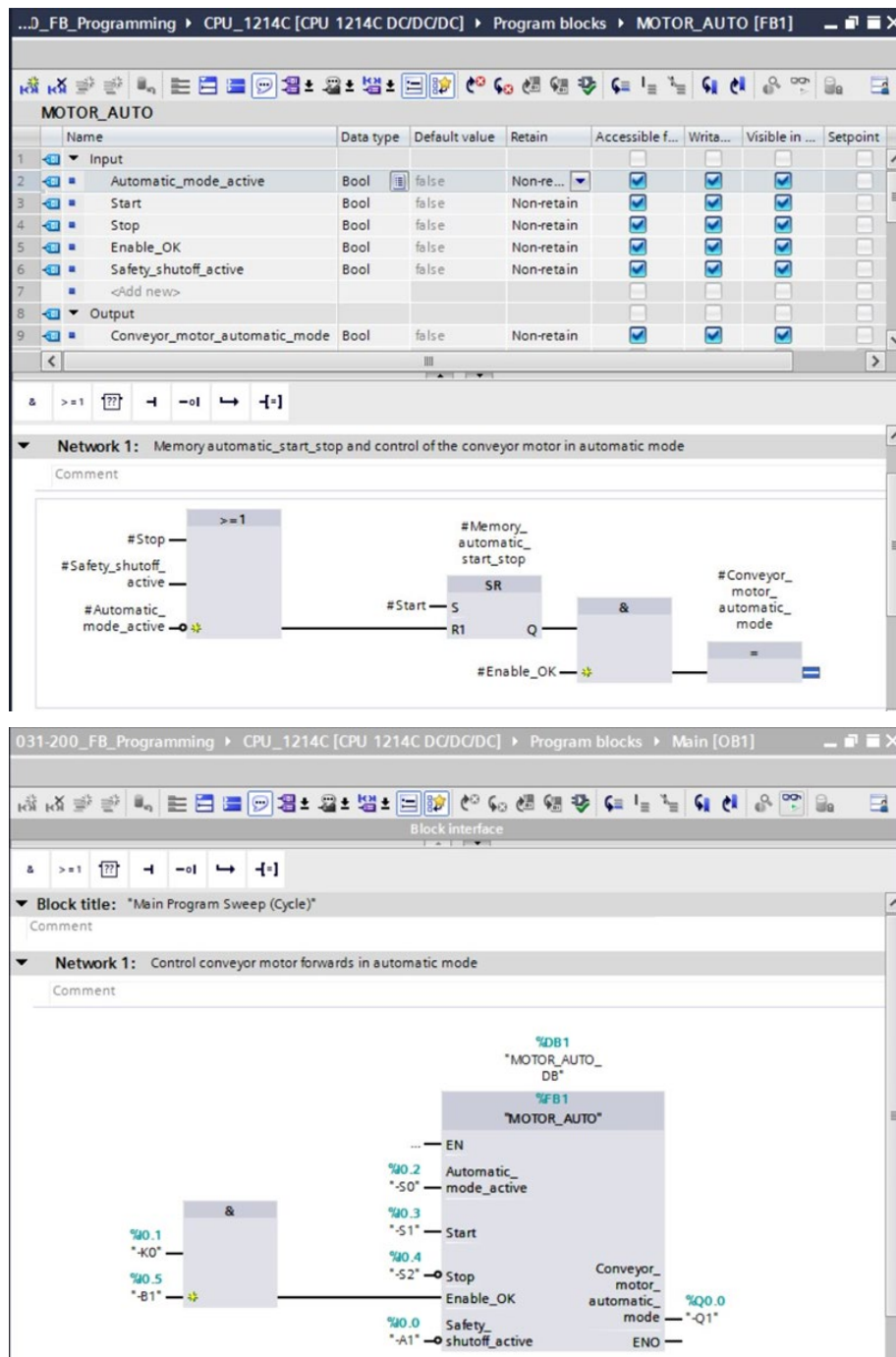


Figure 6: Library-compatible function block with call in OB1

4.8 Programming languages

For SIMATIC S7-1200 controller, the available programming languages for programming functions and function blocks are Function Block Diagram (FBD), Ladder Logic (LAD) and Structured Control Language (SCL).

The **Function Block Diagram (FBD)** programming language will be presented in the following.

FBD is a graphical programming language. The representation is based on electronic switching systems. The program is mapped in networks. A network contains one or more logic operation paths. Binary and analog signals are linked together by boxes. The graphical logic symbols known from Boolean algebra are used to represent the binary logic.

You can use binary functions to query binary operands and to logically combine their signal states. The following instructions are examples of binary functions: "AND operation", "OR operation" and "EXCLUSIVE OR operation". These are shown in Figure 7.

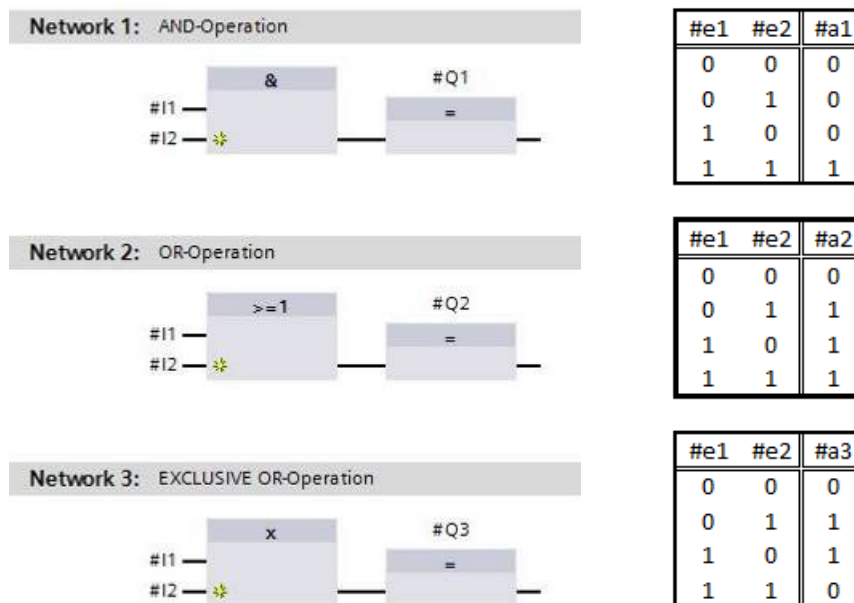


Figure 7: Binary functions in FBD and associated logic table

You can thus use simple instructions, for example, to control binary outputs, evaluate edges and execute jump functions in the program.

Program elements such as IEC timers and IEC counters provide complex instructions.

The empty box serves as a placeholder in which you can select the required instruction.

Enable input EN (enable)/ Enable output ENO (enable output) mechanism:

- An instruction without EN/ENO mechanism is executed independent of the signal state at the box inputs.
- Instructions with EN/ENO mechanism are only executed if enable input "EN input has signal state "1". When the box is processed correctly, enable output "ENO" has signal state "1". If an error occurs during processing, the enable output "ENO" is reset. If enable input EN is not connected, the box is always executed.

5 Task

The following functions of the sorting station process description will be planned, programmed and tested in this chapter:

- Automatic mode - Conveyor motor

6 Planning

The programming of all functions in OB1 is not recommended for reasons of clarity and reusability. The majority of the program code will therefore be moved into functions (FCs) and function blocks (FBs). The decision on which functions are to be moved to the FB and which is to run in OB 1 is planned below.

6.1 EMERGENCY STOP

The EMERGENCY STOP does not require a separate function. Just like the operating mode, the current state of the EMERGENCY STOP relay can be used directly at the blocks.

6.2 Automatic mode - Conveyor motor

Automatic mode of the conveyor motor is to be encapsulated in a function block (FB) "MOTOR_AUTO". On the one hand, this preserves the clarity of OB1. On the other hand, it enables reuse if another conveyor belt is added to the station. Table 2 lists the planned parameters.

Input	Data	Comment
Automatic_mode_active	BOOL	Automatic mode activated
Start	BOOL	Pushbutton automatic start
Stop	BOOL	Pushbutton automatic stop
Enable_OK	BOOL	All enable conditions OK
Safety_shutoff_active	BOOL	Safety shutoff active e.g. emergency stop
Output		
Conveyor_motor_automatic_mode	BOOL	Control of the conveyor motor in automatic mode
Static		
Memory_automatic_start_stop	BOOL	Memory used for start/stop automatic mode

Table 2: Parameters for FB "MOTOR_AUTO"

The Memory_automatic_start_stop is latched with Start but only if the reset conditions are not present.

The Memory_automatic_start_stop is reset if Stop is present or safety shutoff is active or automatic mode is not activated (manual mode).

The Conveyor_motor_automatic_mode output is controlled when Memory_automatic_start_stop is set and the enable conditions are met.

6.3 Technology diagram

Here, you see the technology diagram for the task.

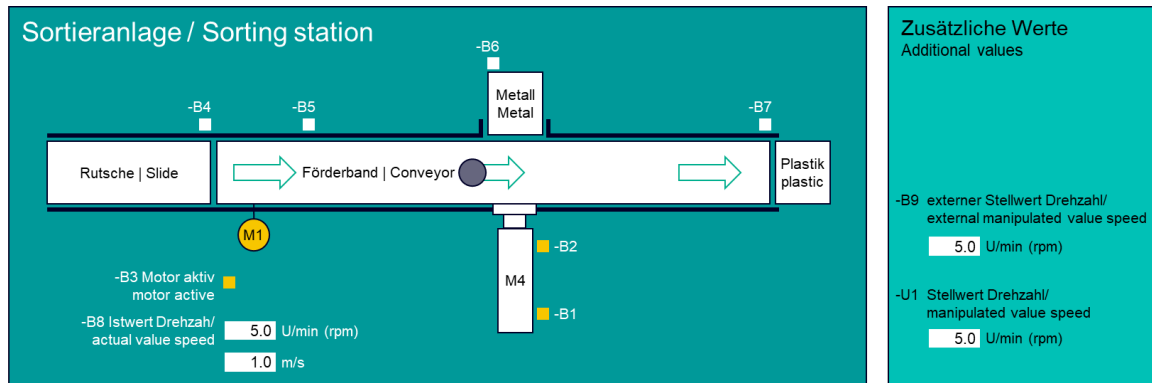


Figure 8: Technology diagram

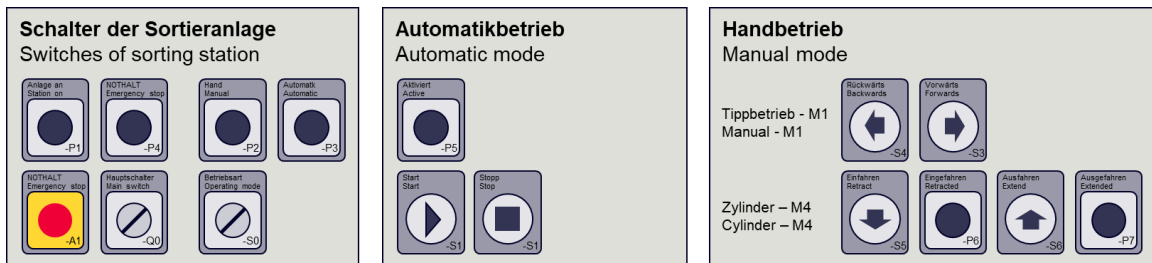


Figure 9: Control panel

6.4 Reference list

The following signals are needed as operands for this task.

DI	Type	Identifier	Function	NC/NO
I 0.0	BOOL	-A1	Return signal emergency stop ok	NC
I 0.1	BOOL	-K0	Main switch "ON"	NO
I 0.2	BOOL	-S0	Mode selector manual (0)/ automatic (1)	Manual = 0 Auto = 1
I 0.3	BOOL	-S1	Pushbutton automatic start	NO
I 0.4	BOOL	-S2	Pushbutton automatic stop	NC
I 0.5	BOOL	-B1	Sensor cylinder M4 retracted	NO

DO	Type	Identifier	Function	
Q 0.0	BOOL	-Q1	Conveyor motor M1 forwards fixed speed	

Legend for reference list

DI Digital Input

DO Digital Output

AI Analog Input

AO Analog Output

I Input

Q Output

NC Normally Closed

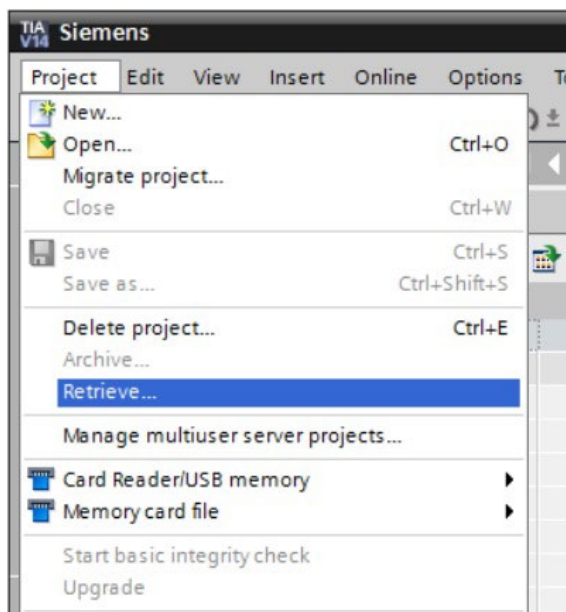
NO Normally Open

7 Structured step-by-step instructions

You can find instructions on how to carry out planning below. If you already have a good understanding of everything, it will be sufficient to focus on the numbered steps. Otherwise, simply follow the detailed steps in the instructions.

7.1 Retrieve an existing project

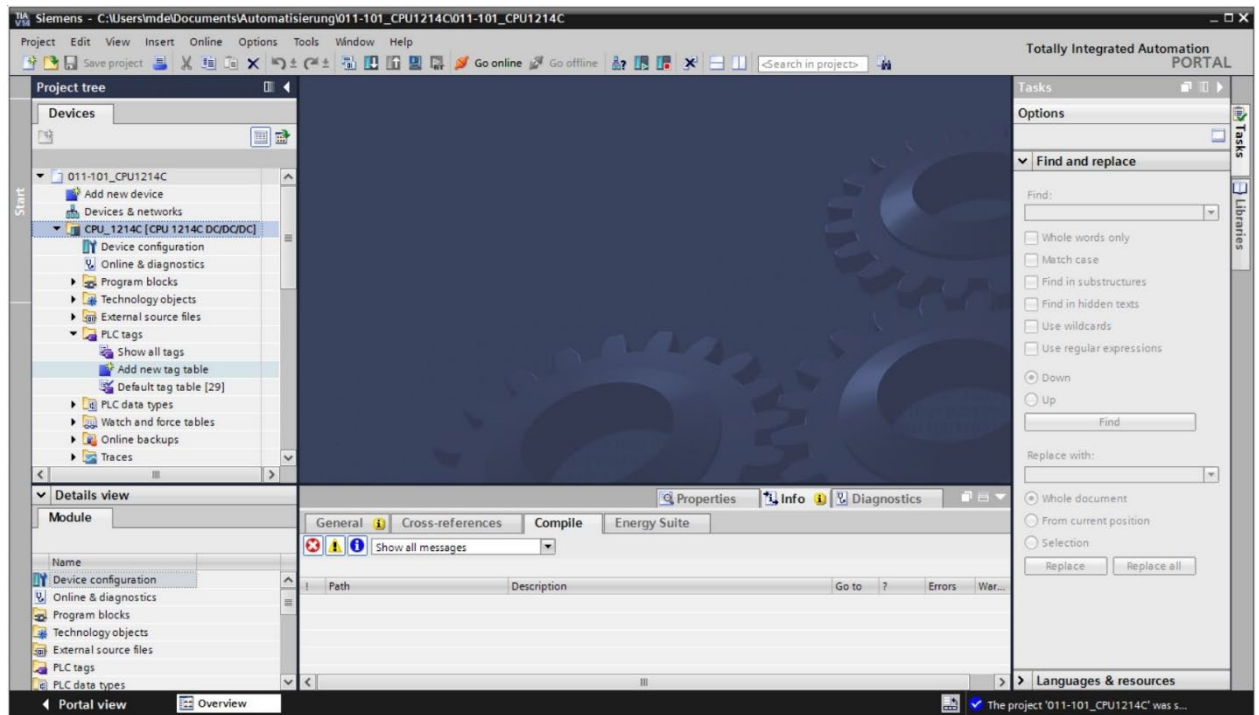
- Before we can start programming the function block (FB) "MOTOR_AUTO", we need a project with a hardware configuration (e.g. SCE_EN_011_101_Hardware_Configuration_S7-CPU1214C....zap). To retrieve an existing project that has been archived, you must select the relevant archive with → Project → Retrieve in the project view. Confirm your selection with Open (→ Project → Retrieve → Select a .zap archive → Open).



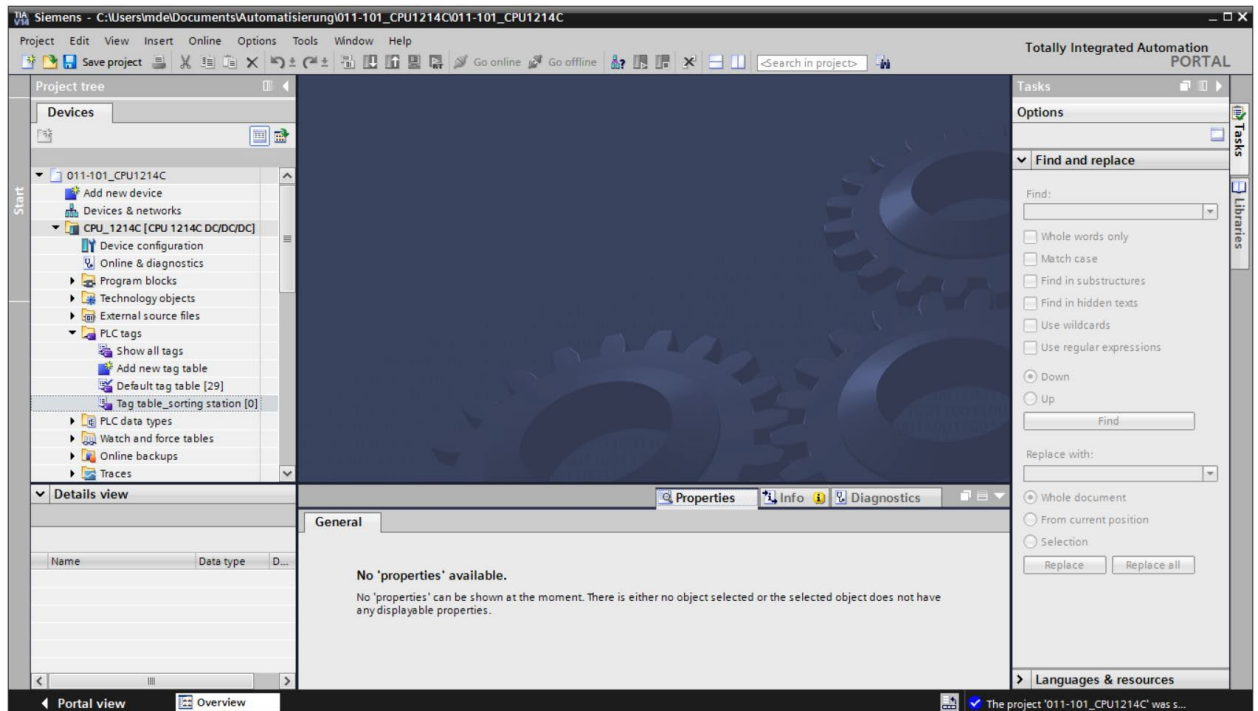
- The next step is to select the target directory where the retrieved project will be stored. Confirm your selection with "OK". (→ Target directory → OK)

7.2 Create a new tag table

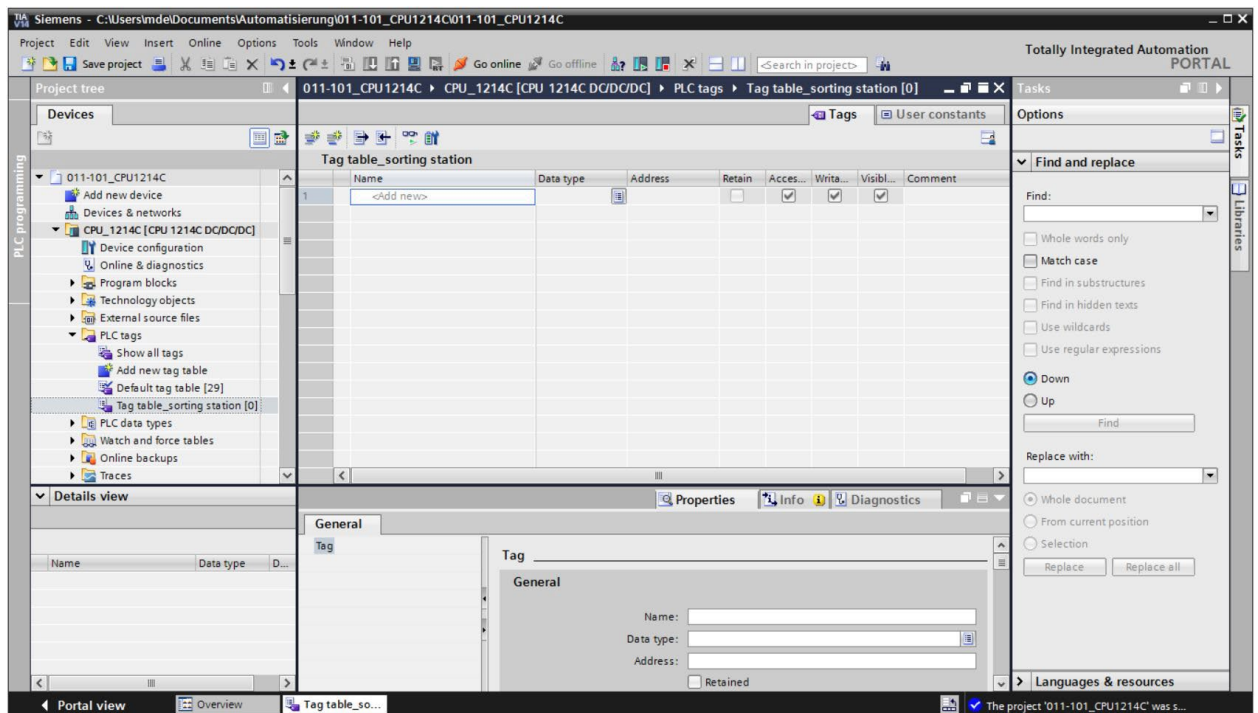
→ In the project view, navigate to the → PLC tags of your controller and create a new tag table by double-clicking → Add new tag table.



→ Rename the tag table you just created as "Tag_table_sorting_station" (→ right-click "Tag_table_1" → "Rename" → Tag_table_sorting_station).

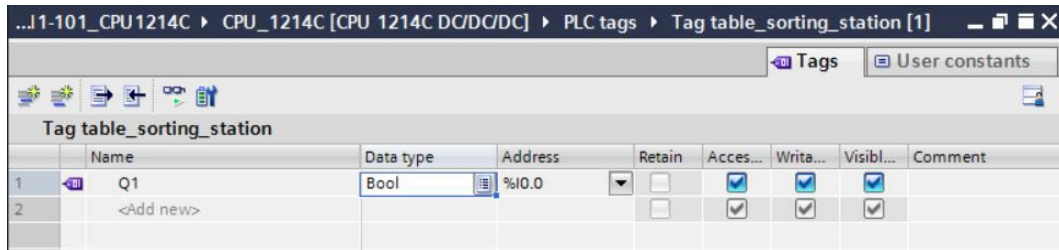


→ Open this tag table with a double-click. (→ Tag_table_sorting_station)

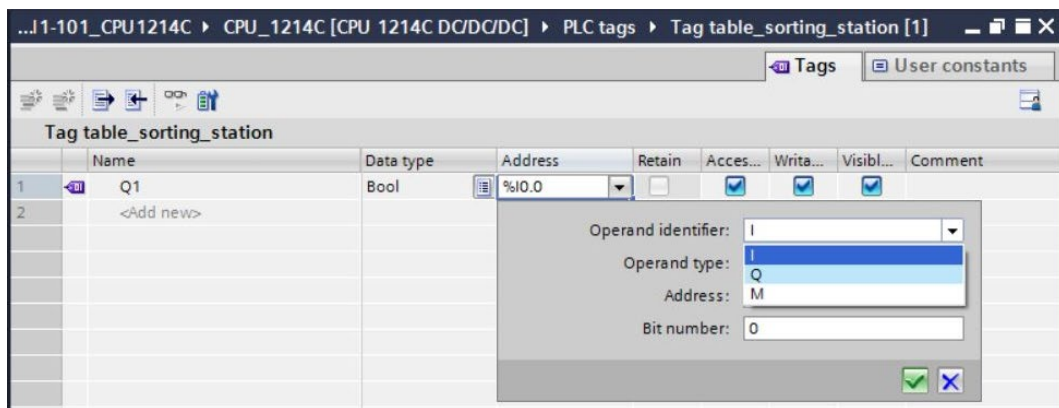


7.3 Create new tags within a tag table

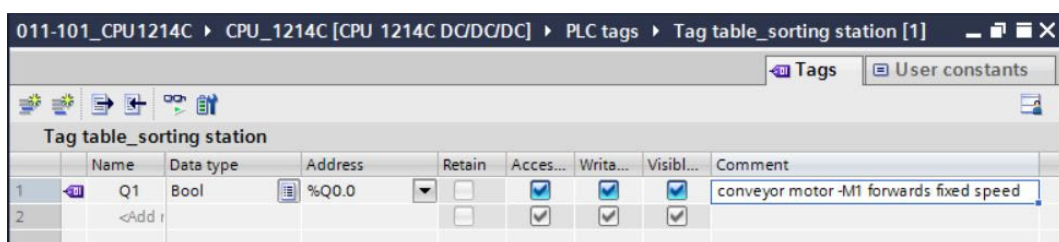
- Add the name Q1 and confirm the entry with the Enter key. If you have not yet created additional tags, TIA Portal now automatically assigns data type "Bool" and address %I0.0 (I 0.0) (→ <Add> → Q1 → Enter).



- Change the address to %Q0.0 (Q 0.0) by entering this directly or by clicking the drop-down arrow to open the Addressing menu. Change the operand identifier to Q and confirm with Enter or by clicking the check mark (→ %I0.0 → Operand identifier→ Q →)

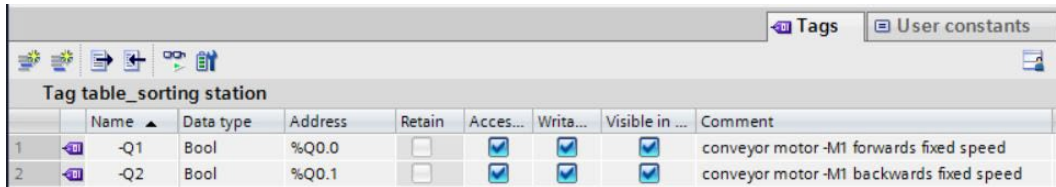


- Enter the "Conveyor motor M1 forwards fixed speed" comment for the tag.



→ Add a new Q2 tag in line 2. TIA Portal has automatically assigned the same data type as the one in line 1 and has incremented the address by 1 to %Q0.1 (Q0.1). Enter the comment "Conveyor motor M1 backwards fixed speed".

(→ <Add> → Q2 → Enter → Comment → Conveyor motor M1 backwards fixed speed)

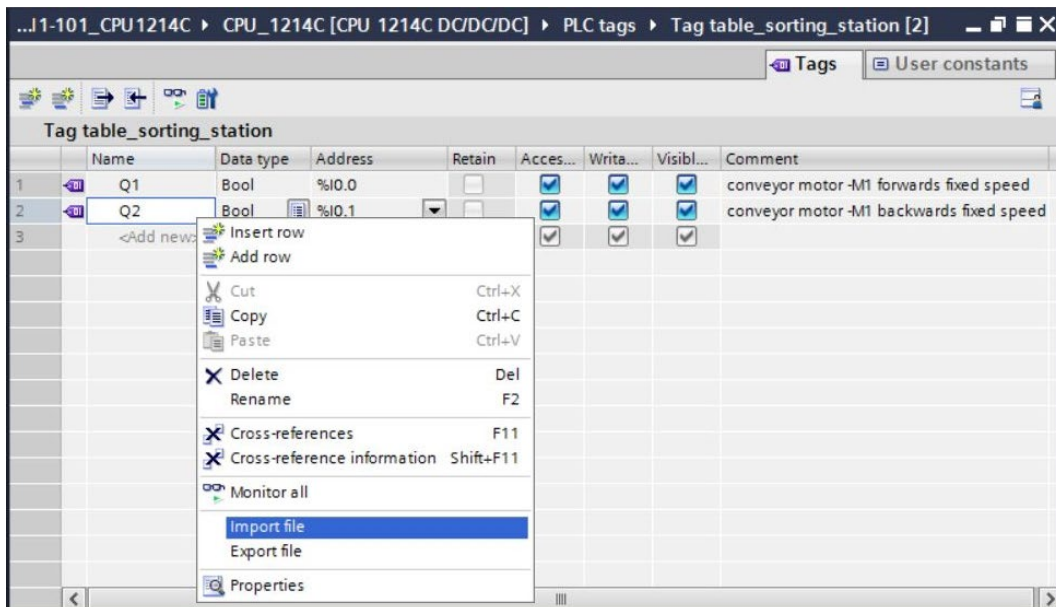


	Name	Data type	Address	Retain	Acces...	Writa...	Visible in ...	Comment
1	-Q1	Bool	%Q0.0	☐	☒	☒	☒	conveyor motor -M1 forwards fixed speed
2	-Q2	Bool	%Q0.1	☐	☒	☒	☒	conveyor motor -M1 backwards fixed speed

7.4 Import "Tag_table_sorting_station"

→ To insert an existing symbol table, right-click on an empty field of the created "Tag_table_sorting_station". Select "Import file" in the shortcut menu.

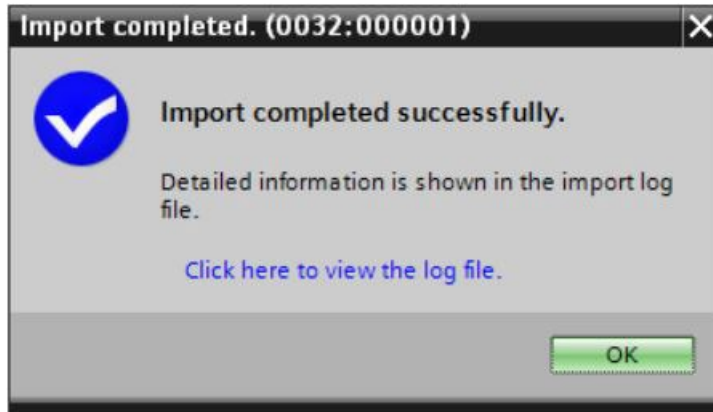
(→ Right-click in an empty field of the tag table → Import file)



→ Select the desired symbol table (e.g. in .xlsx format) and confirm the selection with "Open".

(→ SCE_EN_020-100_Tag_table_sorting_station... → Open)

→ When the import is finished, you will see a confirmation window and have an opportunity to view the log file for the import. Click → OK.



You can see that some addresses have been highlighted in orange. These are duplicate addresses and the names of the associated tags have been numbered automatically to avoid confusion.

→ Delete the duplicate tags by selecting the lines and pressing the Del key on your keyboard or selecting "Delete" in the shortcut menu.

(→ Right-click on selected tags → Delete)

...1-101_CPU1214C > CPU_1214C [CPU 1214C DC/DC/DC] > PLC tags > Tag table_sorting station [30]

Tags User constants

Tag table_sorting station

	Name	Data type	Address	Retain	Acces...	Writa...	Visibl...	Comment
1	Q1	Bool	%Q0.0	☐	☒	☒	☒	conveyor motor -M1 forwards fixed speed
2	Q2	Bool	%Q0.1	☐	☒	☒	☒	conveyor motor -M1 forwards fixed speed
3	-A1	Bool	%I0.0	☐	☒	☒	☒	return signal emergency stop ok (nc)
4	-K0	Bool	%I0.1	☐	☒	☒	☒	main switch „ON“ (no)
5	-S0	Bool	%I0.2	☐	☒	☒	☒	mode selector manual(0) / automatic(1)
6	-S1	Bool	%I0.3	☐	☒	☒	☒	pushbutton automatic start (no)
7	-S2	Bool	%I0.4	☐	☒	☒	☒	pushbutton automatic stop (nc)
8	-B1	Bool	%I0.5	☐	☒	☒	☒	sensor cylinder -M4 retracted (no)
9	-B2	Bool	%I0.6	☐	☒	☒	☒	sensor cylinder -M4 extended (nc)
10	-B3	Bool	%I0.7	☐	☒	☒	☒	sensor motor -M1 actice (pulse signal for ...
11	-B4	Bool	%I1.0	☐	☒	☒	☒	sensor part at slide (no)
12	-B5	Bool	%I1.1	☐	☒	☒	☒	sensor metal part (no)
13	-B6	Bool	%I1.2	☐	☒	☒	☒	sensor part in front of cylinder -M4 (no)
14	-B7	Bool	%I1.3	☐	☒	☒	☒	sensor part at end of conveyor (no)
15	-S3	Bool	%I1.4	☐	☒	☒	☒	pushbutton manual mode conveyor -M1...
16	-S4	Bool	%I1.5	☐	☒	☒	☒	pushbutton manual mode conveyor -M1...
17	-S5	Bool	%I1.6	☐	☒	☒	☒	pushbutton manual mode cylinder -M4 re...
18	-S6	Bool	%I1.7	☐	☒	☒	☒	pushbutton manual mode cylinder -M4 ex...
19	-Q1	Bool	%Q0.0	☐	☒	☒	☒	conveyor motor -M1 forwards fixed speed
20	-Q2	Bool	%Q0.1	☐	☒	☒	☒	conveyor motor -M1 backwards fixed speed
21	-Q3	Bool	%Q0.2	☐	☒	☒	☒	conveyor motor -M1 variable speed
22	-M2	Bool	%Q0.3	☐	☒	☒	☒	cylinder -M4 retract
23	-M3	Bool	%Q0.4	☐	☒	☒	☒	cylinder -M4 extend
24	-P1	Bool	%Q0.5	☐	☒	☒	☒	display „main switch on“
25	-P2	Bool	%Q0.6	☐	☒	☒	☒	display „manual mode“
26	-P3	Bool	%Q0.7	☐	☒	☒	☒	display „automatic mode“

→ You now have a complete symbol table of the digital inputs and outputs in front of you.

Save your project under the name 031-200_FB_Programming.

(→ Project → Save as ... → 031-200_FB_Programming → Save)

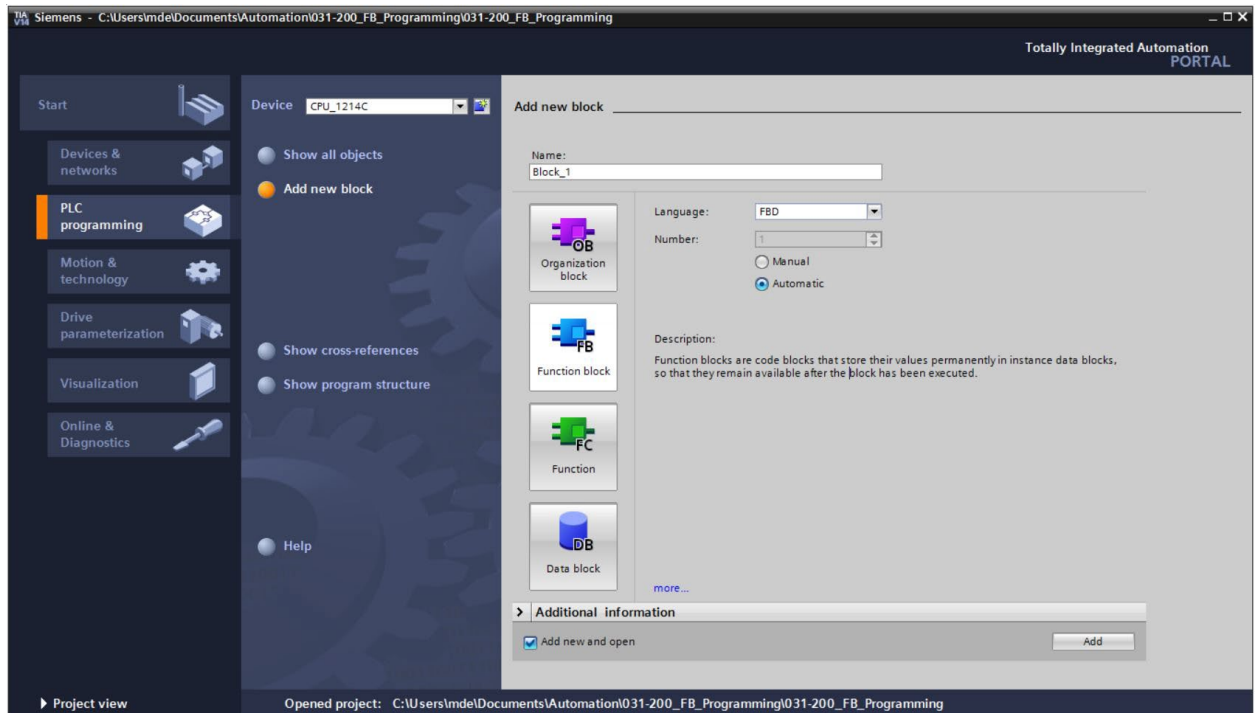
The screenshot shows the Siemens TIA Portal software interface. The main window displays the 'Tag table_sorting station [28]' table. The table has columns for Name, Data type, Address, Retain, Access, Write, Visible, and Comment. The table lists various digital inputs and outputs, including emergency stop signals, main switch, mode selector, pushbutton, sensor, and motor signals.

Name	Data type	Address	Retain	Access	Write	Visible	Comment
-A1	Bool	%I0.0					return signal emergency stop ok (nc)
-K0	Bool	%I0.1					main switch „ON“ (no)
-S0	Bool	%I0.2					mode selector manual(0) / automatic(1)
-S1	Bool	%I0.3					pushbutton automatic start (no)
-S2	Bool	%I0.4					pushbutton automatic stop (nc)
-B1	Bool	%I0.5					sensor cylinder -M4 retracted (no)
-B2	Bool	%I0.6					sensor cylinder -M4 extended (nc)
-B3	Bool	%I0.7					sensor motor -M1 actice (pulse signal for ...)
-B4	Bool	%I1.0					sensor part at slide (no)
-B5	Bool	%I1.1					sensor metal part (no)
-B6	Bool	%I1.2					sensor part in front of cylinder -M4 (no)
-B7	Bool	%I1.3					sensor part at end of conveyor (no)
-S3	Bool	%I1.4					pushbutton manual mode conveyor -M1...
-S4	Bool	%I1.5					pushbutton manual mode conveyor -M1...
-S5	Bool	%I1.6					pushbutton manual mode cylinder -M4 re...
-S6	Bool	%I1.7					pushbutton manual mode cylinder -M4 ex...
-Q1	Bool	%Q0.0					conveyor motor -M1 forwards fixed speed
-Q2	Bool	%Q0.1					conveyor motor -M1 backwards fixed speed
-Q3	Bool	%Q0.2					conveyor motor -M1 variable speed
-M2	Bool	%Q0.3					cylinder -M4 retract
-M3	Bool	%Q0.4					cylinder -M4 extend
-P1	Bool	%Q0.5					display „main switch on“
-P2	Bool	%Q0.6					display „manual mode“
-P3	Bool	%Q0.7					display „automatic mode“
-P4	Bool	%Q1.0					display „emergency stop activated“
-P5	Bool	%Q1.1					display „automatic mode started“
-P6	Bool	%Q1.2					display cylinder -M4 „retracted“

7.5 Create function block FB1 "MOTOR_AUTO" for the conveyor motor in automatic mode

→ In the PLC programming section of the portal view, click "Add new block" to create a new

function block. (→ PLC programming → Add new block → )



→ Rename your new block to: "MOTOR_AUTO", set the language to FBD and keep automatic assignment of the number. Select the "Add new and open" check box. You will thus be taken automatically to your created function block in the project view. Click "Add".

(→ Name: MOTOR_AUTO → Language: FBD → Number: Automatic → ☒ Add new and open → Add)

Add new block

Name:

Organization block

Function block

Function

Data block

[more...](#)

Language:

Number:

☐ Manual

☒ Automatic

Description:

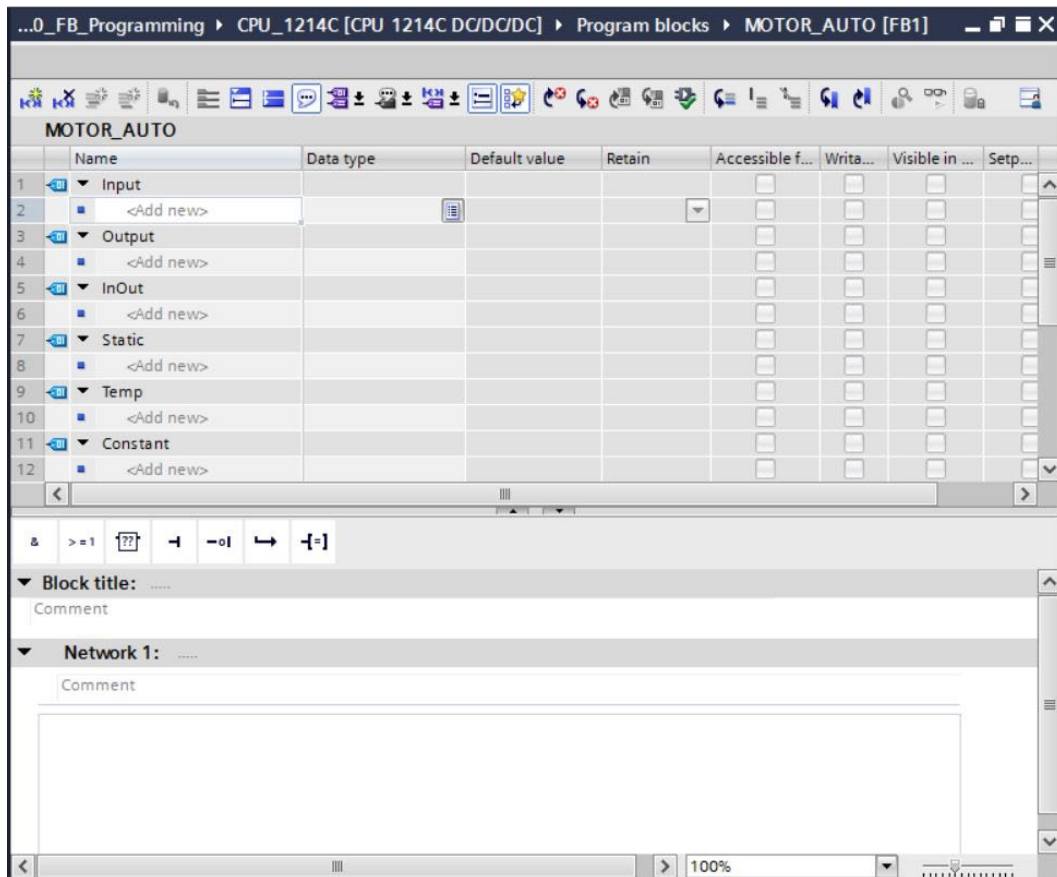
Function blocks are code blocks that store their values permanently in instance data blocks, so that they remain available after the block has been executed.

> Additional information

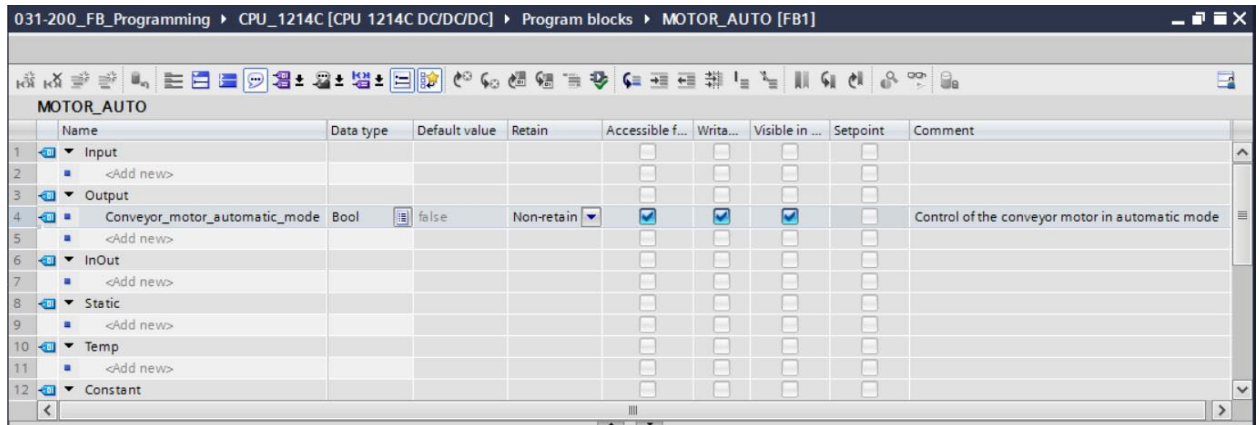
☒ Add new and open

7.6 Define the interface of FB1 "MOTOR_AUTO"

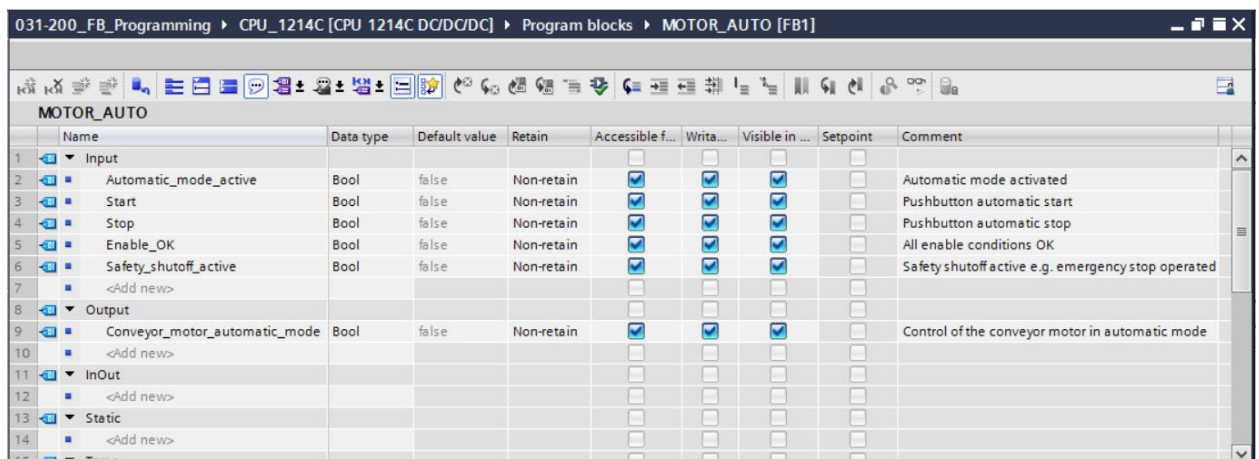
- If you selected "Add new and open", the project view opens with a window for creating the block you just added.
- You can find the interface description of your function block in the upper section of your programming view.



- A binary output signal is needed for controlling the conveyor motor. For this reason, we first create local output tag #Conveyor_motor_automatic_mode of the "Bool" type. Enter the comment "Control of the conveyor motor in automatic mode" for the parameter.
(→ Output → Conveyor_motor_automatic_mode → Bool → Control of the conveyor motor in automatic mode)



- Add parameter #Automatic_mode_active as the input interface under Input and confirm the entry with the Enter key or by exiting the entry field. Data type "Bool" is assigned automatically. This will be retained. Next, enter the associated comment "Automatic mode activated".
(→ Input → Automatic_mode_active → Bool → Automatic mode activated)
- Continue by adding parameters #Start, #Stop, #Enable_OK and #Safety_shutoff_active as additional binary input parameters and check their data types. Add descriptive comments.



- The conveyor is started and stopped with pushbuttons. We therefore need a "Static" tag as a memory. Under Static, add tag #Memory_automatic_start_stop and confirm the entry with the Enter key or by exiting the entry field. Data type "Bool" is assigned automatically. This will be retained. Enter the associated comment "Memory used for start_stop automatic mode".
- (→ Static → Memory_automatic_start_stop → Bool → Memory used for start/stop automatic mode)

031-200_FB_Programming ▶ CPU_1214C [CPU 1214C DC/DC/DC] ▶ Program blocks ▶ MOTOR_AUTO [FB1]

MOTOR_AUTO

	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	Comment
1	▼ Input								
2	Automatic_mode_active	Bool	false	Non-retain	☒	☒	☒	☐	Automatic mode activated
3	Start	Bool	false	Non-retain	☒	☒	☒	☐	Pushbutton automatic start
4	Stop	Bool	false	Non-retain	☒	☒	☒	☐	Pushbutton automatic stop
5	Enable_OK	Bool	false	Non-retain	☒	☒	☒	☐	All enable conditions OK
6	Safety_shutoff_active	Bool	false	Non-retain	☒	☒	☒	☐	Safety shutoff active e.g. emergency stop operated
7	<Add new>				☐	☐	☐	☐	
8	▼ Output								
9	Conveyor_motor_automatic_mode	Bool	false	Non-retain	☒	☒	☒	☐	Control of the conveyor motor in automatic mode
10	<Add new>				☐	☐	☐	☐	
11	▼ InOut								
12	<Add new>				☐	☐	☐	☐	
13	▼ Static								
14	Memory_automatic_start_stop	Bool	false	Non-retain	☒	☒	☒	☐	Memory used for start/ stop automatic mode
15	<Add new>				☐	☐	☐	☐	
16	▼ Temp								
17	<Add new>				☐	☐	☐	☐	
18	▼ Constant								
19	<Add new>				☐	☐	☐	☐	

- For purposes of program documentation, assign the block title, a block comment and a helpful network title for Network 1.

(→ Block title: Motor control in automatic mode → Network 1: Memory_automatic_start_stop and control of the conveyor motor in automatic mode)

031-200_FB_Programming ▶ CPU_1214C [CPU 1214C DC/DC/DC] ▶ Program blocks ▶ MOTOR_AUTO [FB1]

MOTOR_AUTO

	Name	Data type	Default value	Retain	Accessible f...	Writa...	Visible in ...	Setpoint	Comment
1	▼ Input								
2	Automatic_mode_active	Bool	false	Non-retain	☒	☒	☒	☐	Automatic mode activated
3	Start	Bool	false	Non-retain	☒	☒	☒	☐	Pushbutton automatic start
4	Stop	Bool	false	Non-retain	☒	☒	☒	☐	Pushbutton automatic stop
5	Enable_OK	Bool	false	Non-retain	☒	☒	☒	☐	All enable conditions OK
6	Safety_shutoff_active	Bool	false	Non-retain	☒	☒	☒	☐	Safety shutoff active e.g. emergency stop operated
7	<Add new>				☐	☐	☐	☐	
8	▼ Output								
9	Conveyor_motor_automatic_mode	Bool	false	Non-retain	☒	☒	☒	☐	Control of the conveyor motor in automatic mode
10	<Add new>				☐	☐	☐	☐	
11	▼ InOut								
12	<Add new>				☐	☐	☐	☐	
13	▼ Static								
14	Memory_automatic_start_stop	Bool	false	Non-retain	☒	☒	☒	☐	Memory used for start/ stop automatic mode
15	<Add new>				☐	☐	☐	☐	
16	▼ Temp								
17	<Add new>				☐	☐	☐	☐	
18	▼ Constant								
19	<Add new>				☐	☐	☐	☐	

& >=1 <- <-01 <- <-1

▼ **Block title:** Motor control in automatic mode

▼ Conveyor motor in automatic mode:
 The bit Memory_automatic_start_stop is set with the input Start, but only if the reset conditions are not fulfilled.
 The bit Memory_automatic_start_stop is reset with the input Stop or if the safety shutoff is activated or if the automatic mode is not activated (manual mode).
 If Memory_automatic_start_stop is set, the enable conditions are granted and Memory_conveyor_start_stop is set the output Conveyor_motor_automatic_mode is activated.

▼ **Network 1:** Memory automatic_start_stop and control of the conveyor motor in automatic mode

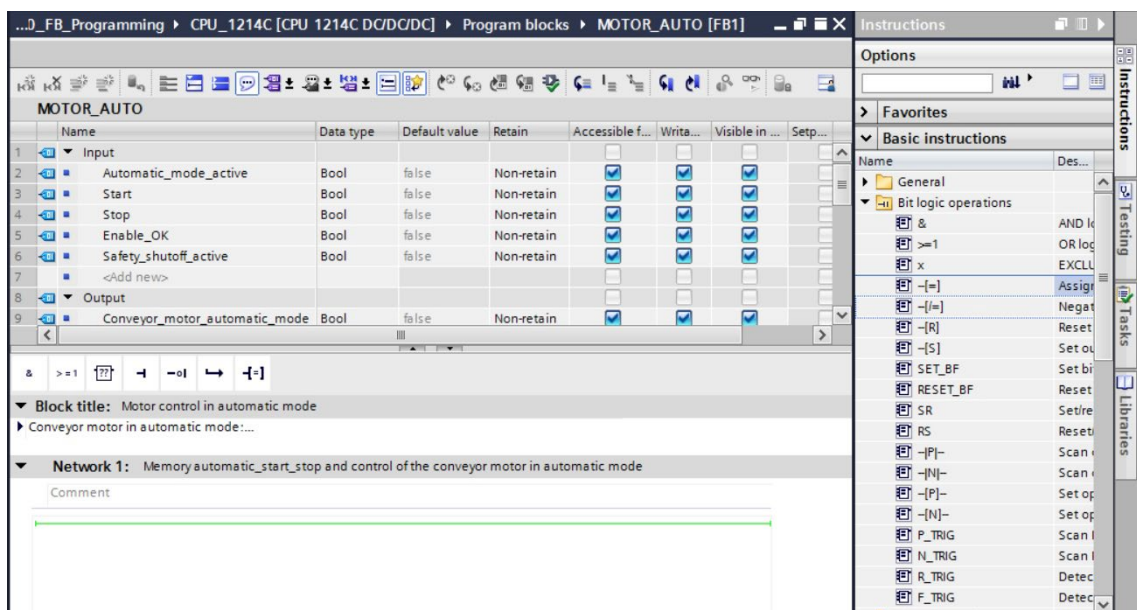
7.7 Program FB1: MOTOR_AUTO

→ Below the interface description, you see a toolbar in the programming window with various logic functions and below that an area with networks. We have already specified the block title and the title for the first network there. Programming is performed within the networks using individual logic blocks. Distribution among multiple networks helps to preserve the clarity of the program. In the following, you will get to know the various ways you can insert logic blocks.

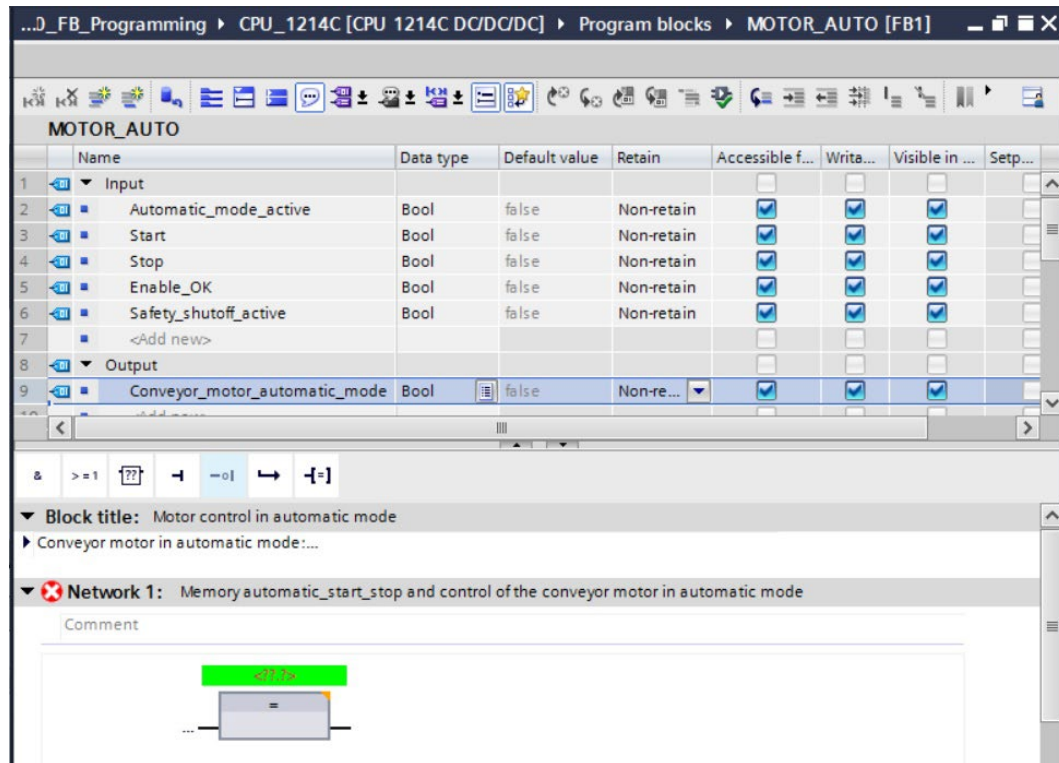


→ On the right side of your programming window is a list of instructions you can use in the program. Under → Basic instructions → Bit logic operations, find function --[=] (Assignment) and use a drag & drop operation to move it to Network 1 (green line appears, mouse pointer with + symbol).

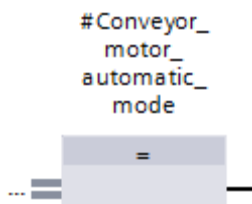
(→ Instructions → Basic instructions → Bit logic operations → --[=])



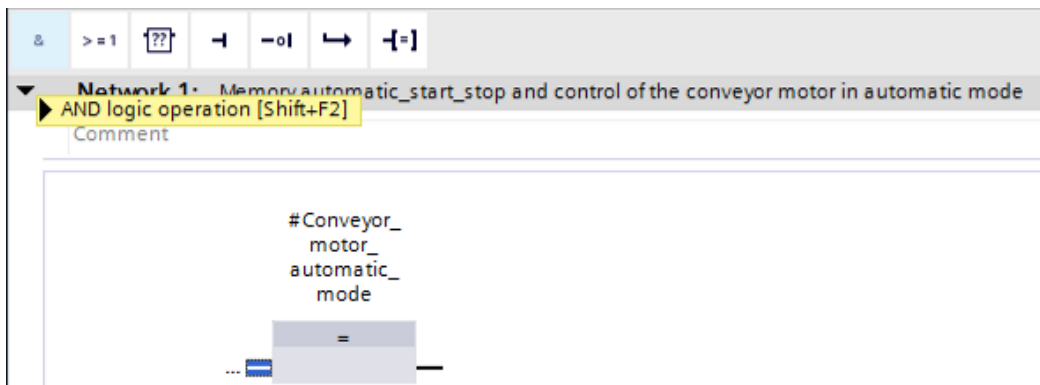
Now use drag & drop to move your output parameter `#Conveyor_motor_automatic_mode` onto `<???.?>` above the block you just inserted. The best way to select a parameter in the interface description is by "grabbing" it at the blue symbol . (→ `Conveyor_motor_automatic_mode`)



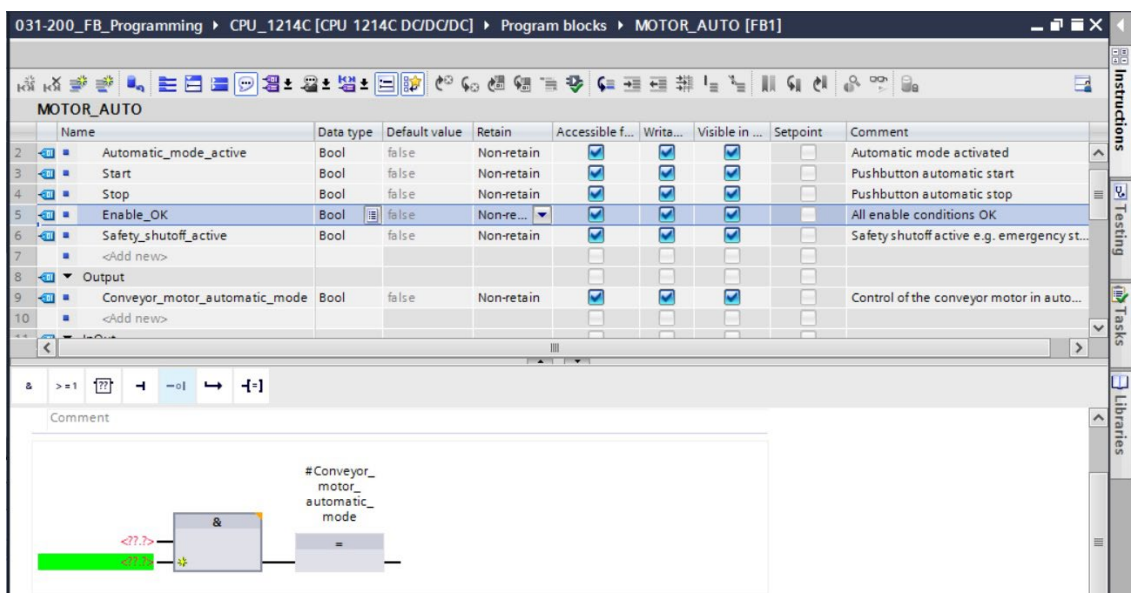
→ This determines that the `#Conveyor_motor_automatic_mode` parameter is written by this block. Still missing, however, are the input conditions so that this actually happens. An SR flip-flop and `#Enable_OK` parameter are logically combined with an AND logic operation at the input of the assignment block. To do this, first click the input of the block so that the input line has a blue background.



→ Click the  icon in your logic toolbar to insert an AND logic operation before your assignment block.

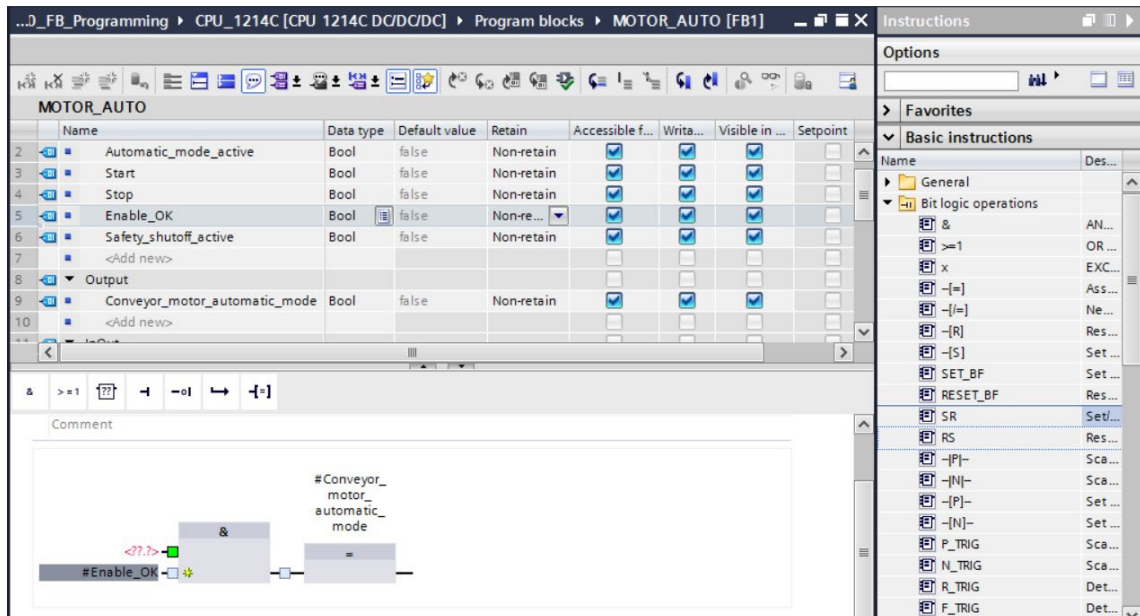


→ Use drag & drop to move input parameter #Enable_OK onto the second input of the & logic operation <??.?>. (→  Enable_OK)



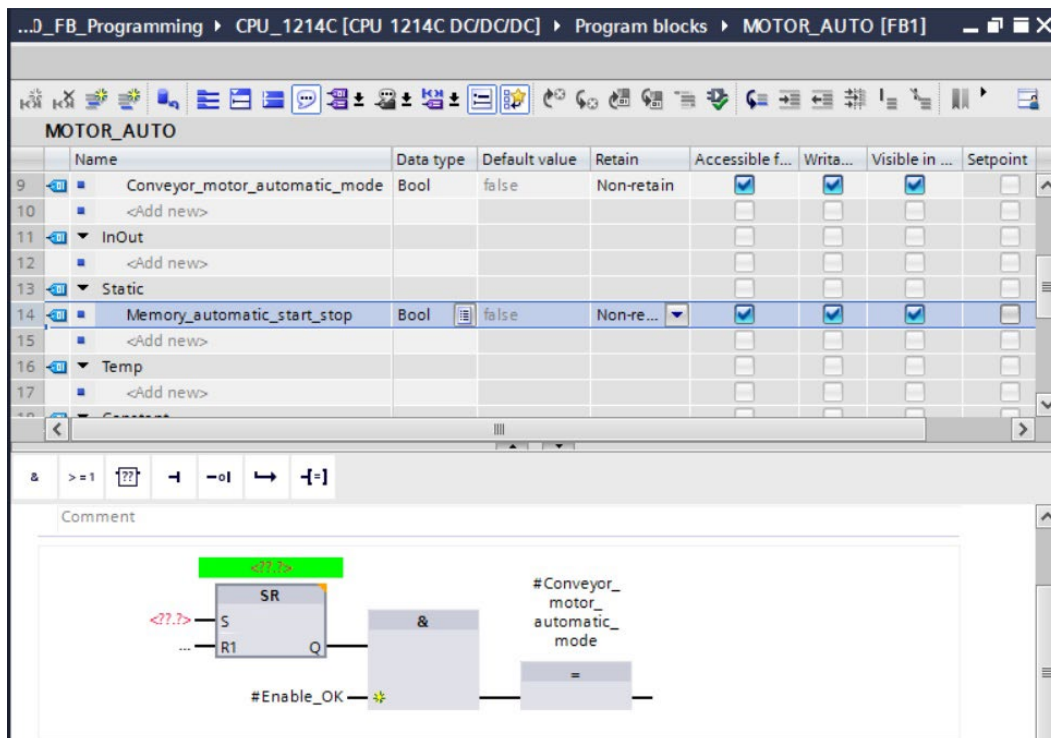
→ Use drag & drop to move the Set/reset flip-flop  from the list of instructions under → Basic instructions → Bit logic operations onto the first input of the & operation .

(→ Instructions → Basic instructions → Bit logic operations →  → )

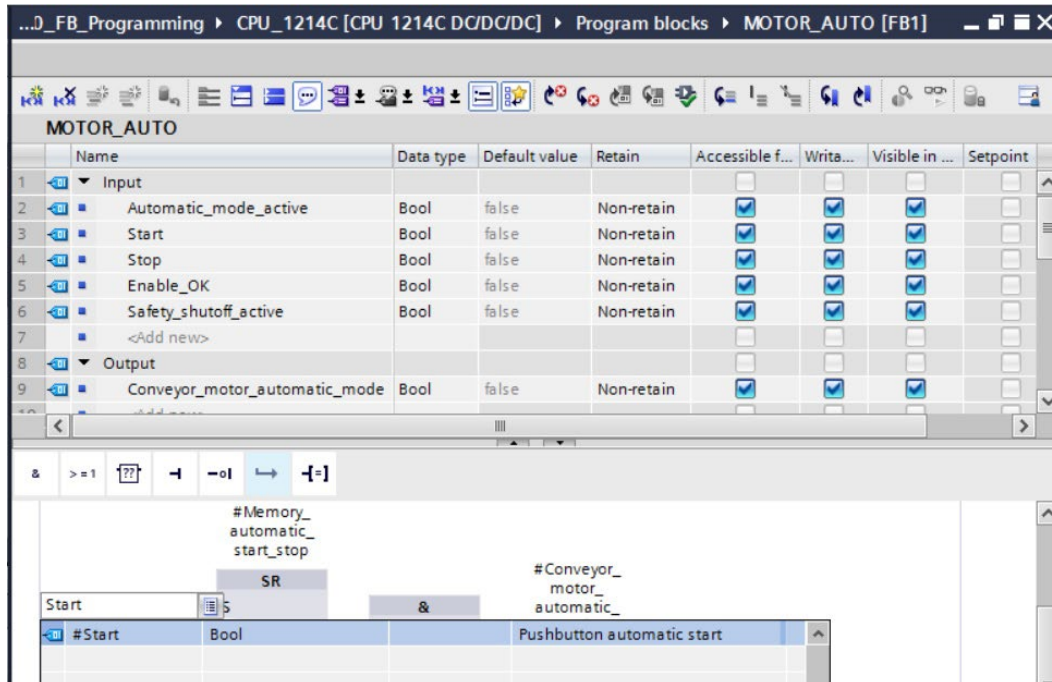


→ The SR flip-flop requires a memory tag. For this, use drag & drop to move static parameter #Memory_automatic_start_stop onto the  above the SR flip-flop.

(→  Memory_automatic_start_stop)



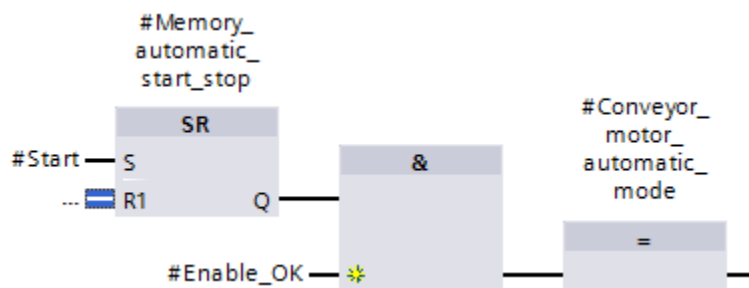
- The #Memory_automatic_start_stop will be set with input tag #Start. Click twice on the S input of the SR flip-flop <???.?> and enter "Start" in the field that appears in order to see a list of available tags starting with "Start". Click the #Start tag and apply with → Enter.
(→ SR flip-flop → <???.?> → Start → #Start → Enter)



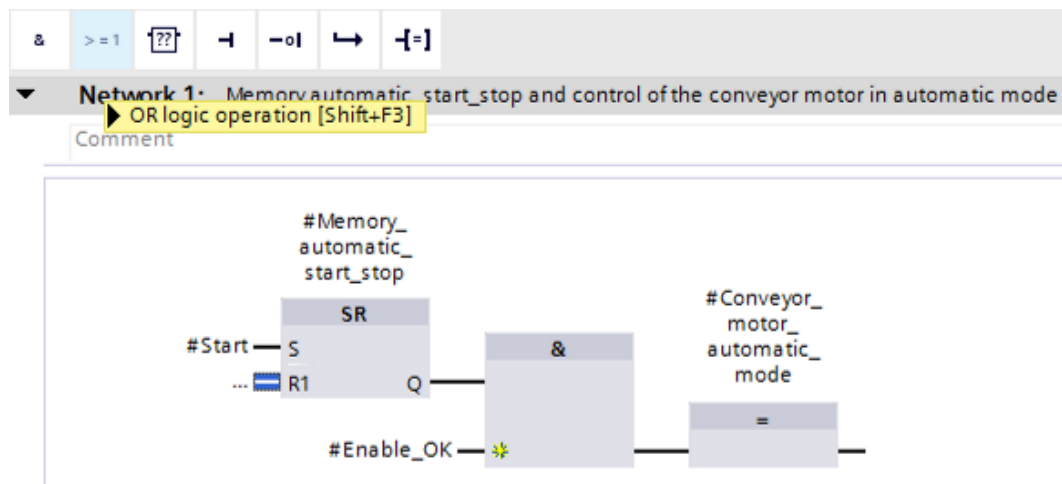
Note:

- When assigning tags in this way, there is a risk of a mix-up with the global tags from the tag table. The previously presented procedure using drag & drop from the interface description should therefore be used preferentially.

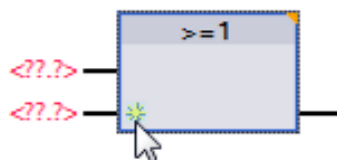
- Multiple conditions are to be able to stop the conveyor. An OR block is therefore needed at the R1 input of the SR flip-flop. First, click the R1 input of the SR flip-flop so that the input line has a blue background.



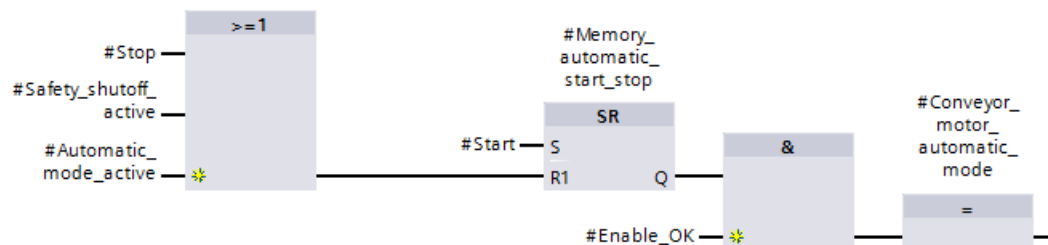
→ Click the  icon in your logic toolbar to insert an OR logic operation.



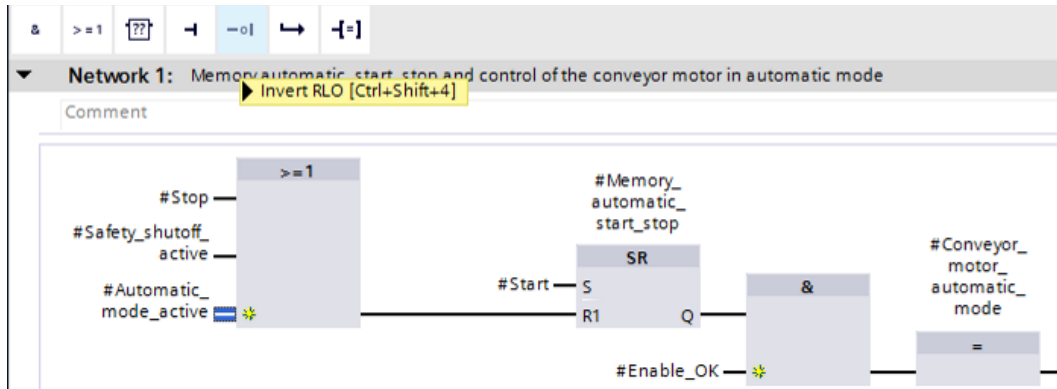
→ The OR block has 2 inputs initially. In order to logically combine an additional input tag, click the yellow star  of your OR block.



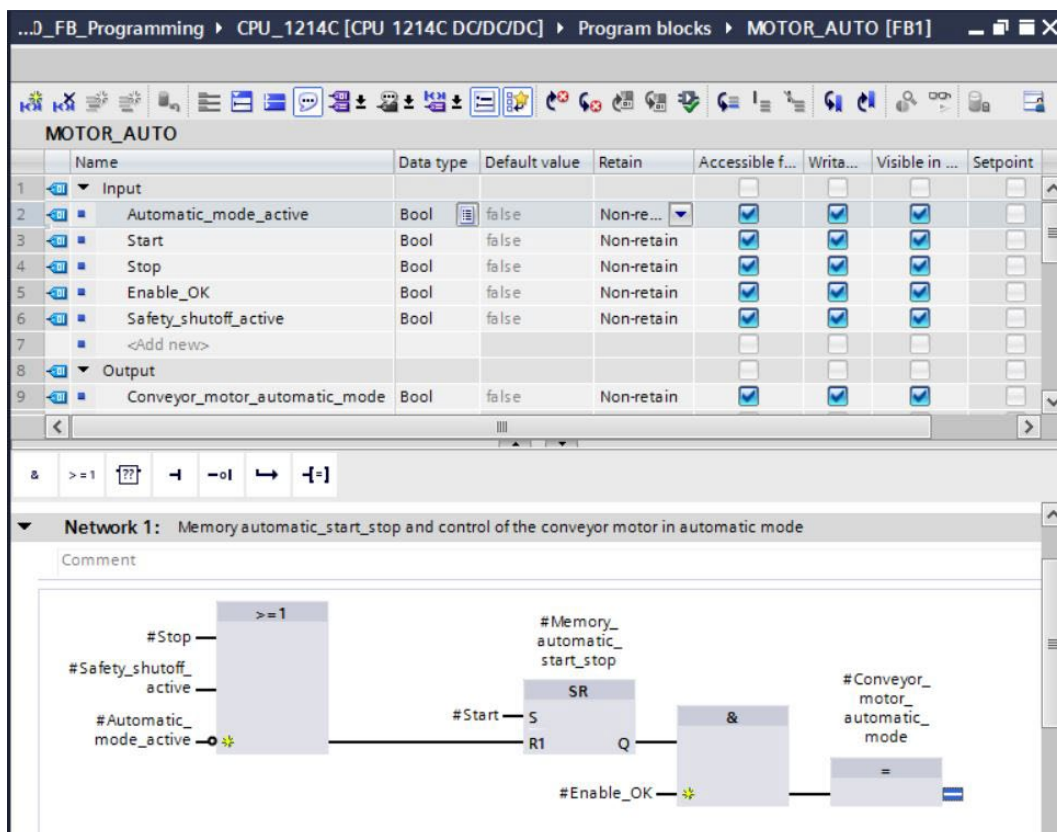
→ Add input tags `#Stop`, `#Safety_shutoff_active` and `#Automatic_mode_active` to the 3 inputs of the OR block.



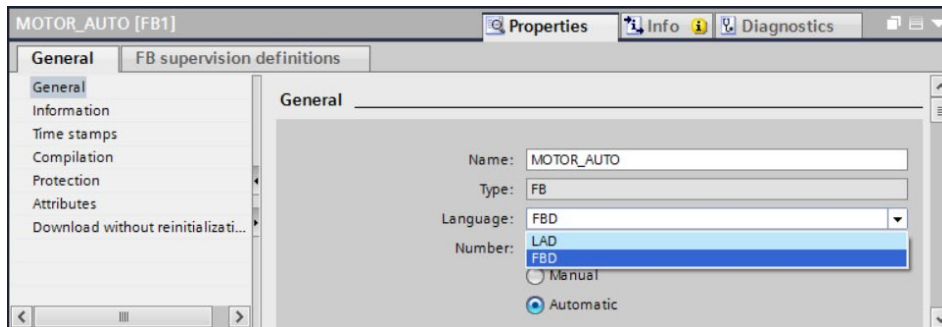
→ Negate the input connected to parameter #Automatic_mode_active by selecting it and clicking .



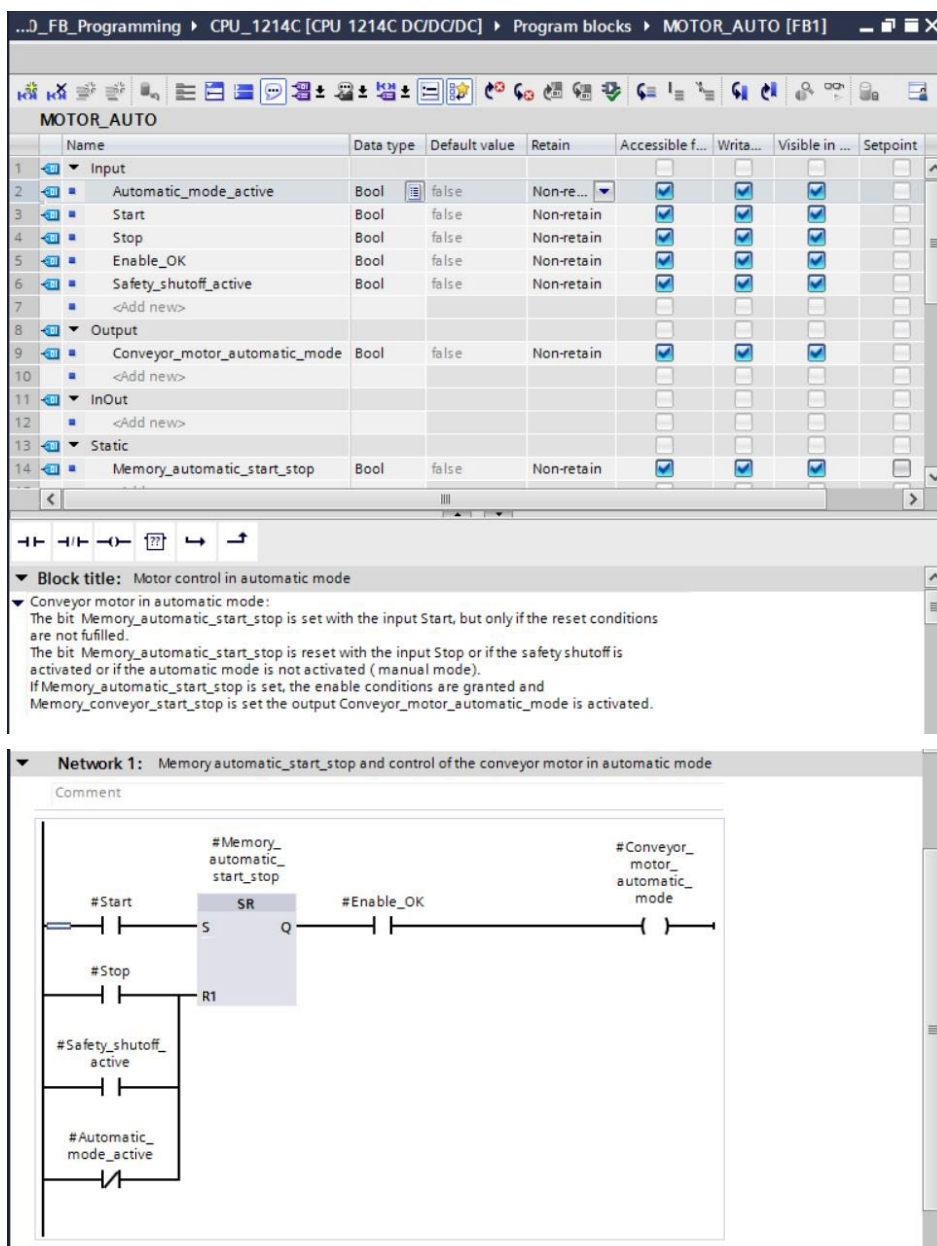
→ Do not forget to click  Save project regularly. The finished function block "MOTOR_AUTO" [FB1] in FBD is shown below.



→ Under "General" in the properties of the block, you can change the "Language" to LAD (Ladder Logic) (→ Properties → General → Language: LAD)



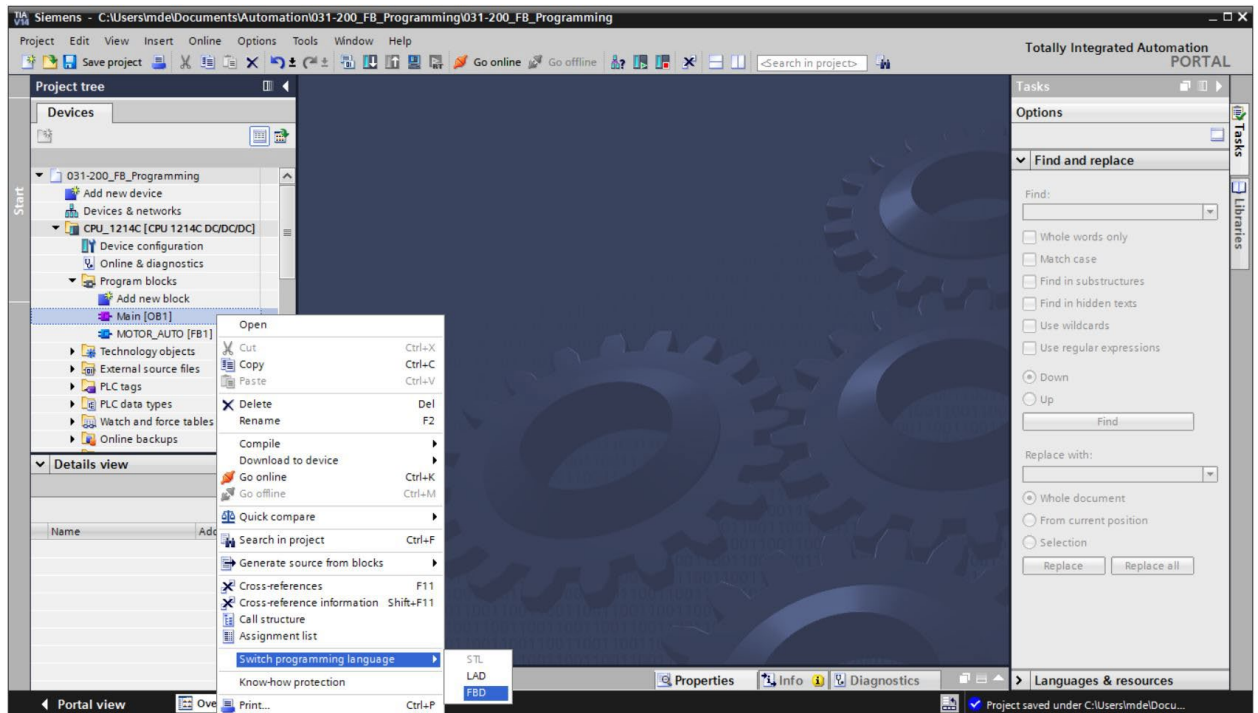
→ The program has the following appearance in LAD.



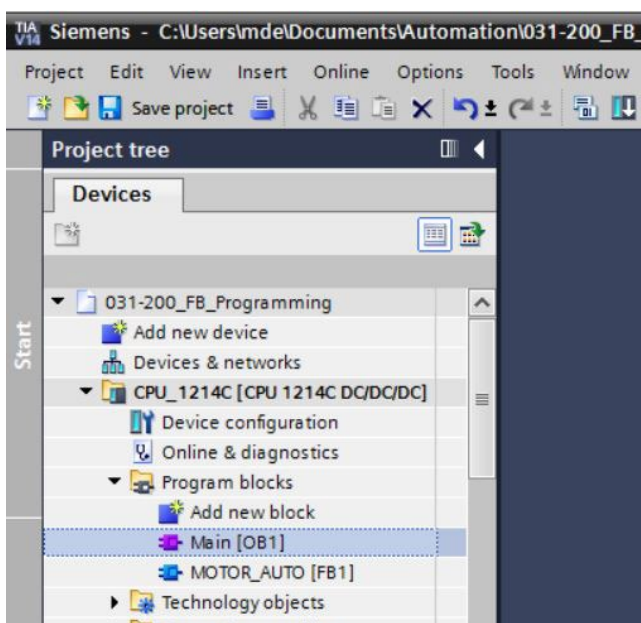
7.8 Program the organization block OB1 – Control conveyor tracking forwards in automatic mode

→ Before programming organization block "Main [OB1]", we switch the programming language to FBD (Function Block Diagram). To do so, first click on "Main [OB1]" in the "Program blocks" folder.

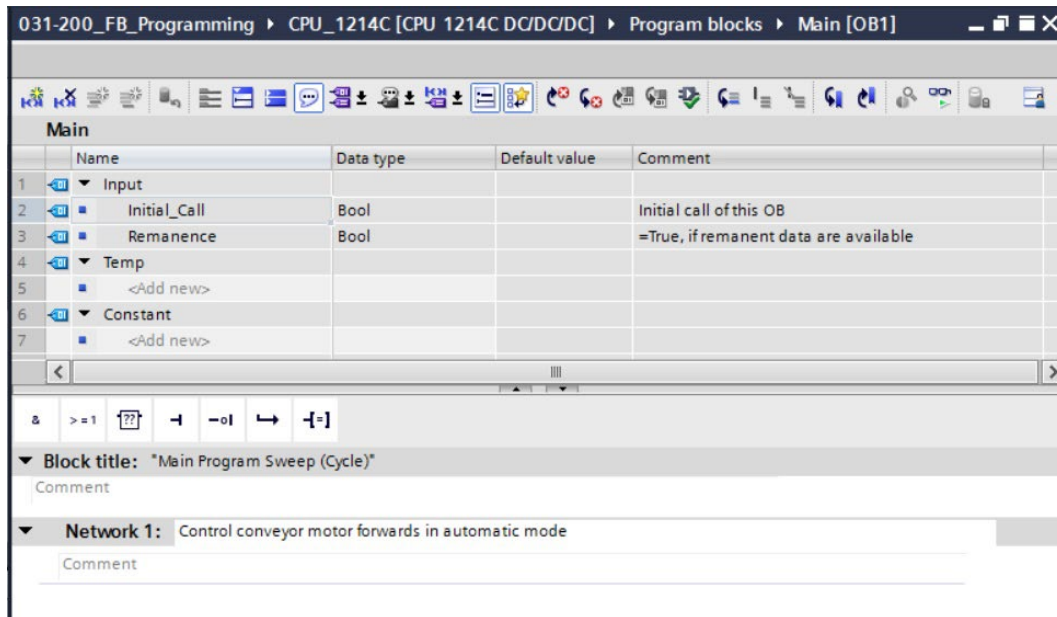
(→ CPU_1214C [CPU 1214C DC/DC/DC] → Program blocks → Main [OB1] → Switch programming language → FBD)



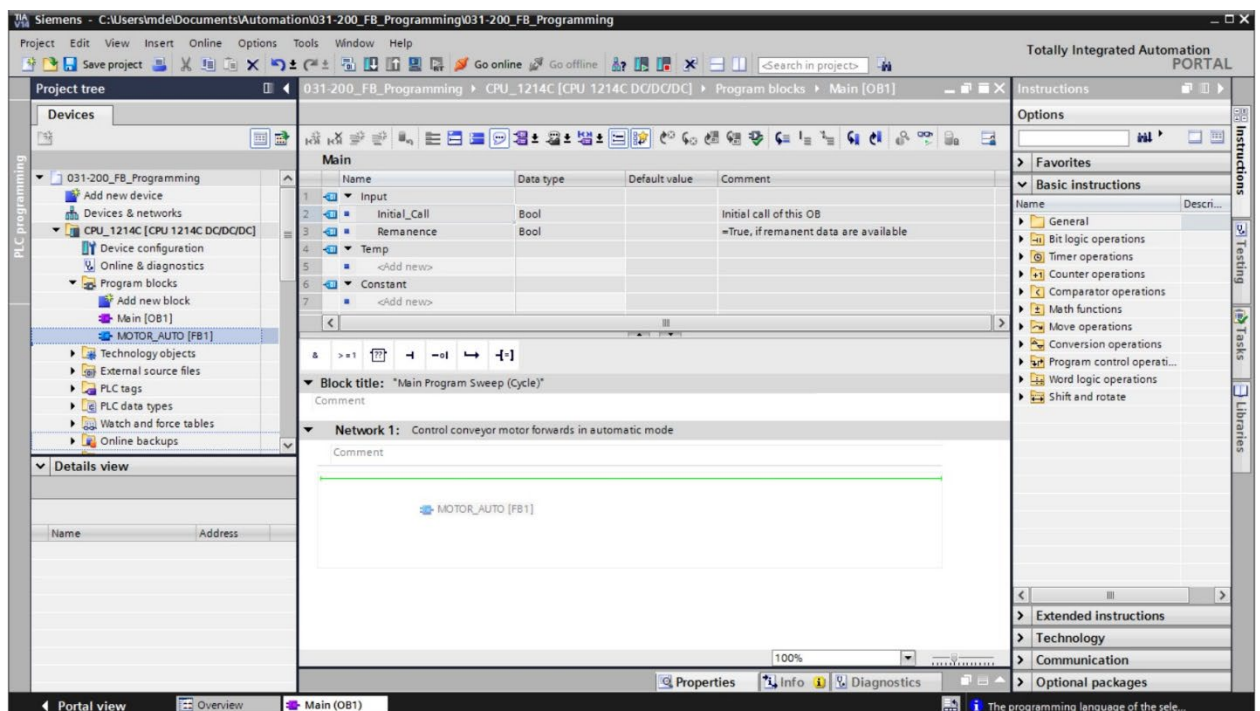
→ Open the "Main [OB1]" organization block with a double-click.



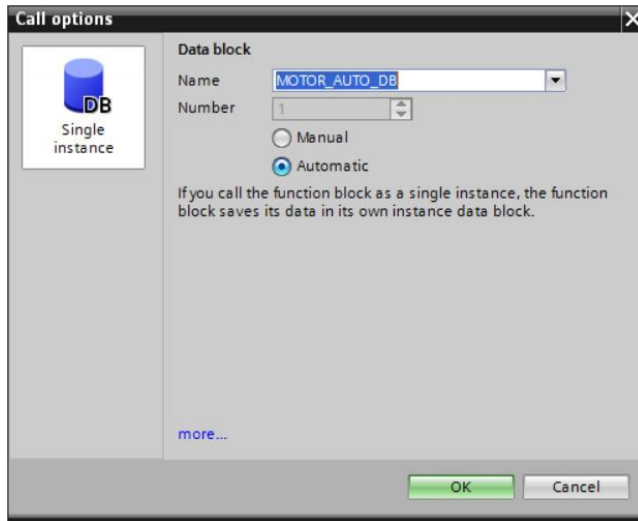
- Assign Network 1 the name "Control conveyor tracking forwards in automatic mode"
 (→ Network 1:... →Control conveyor tracking forwards in automatic mode)



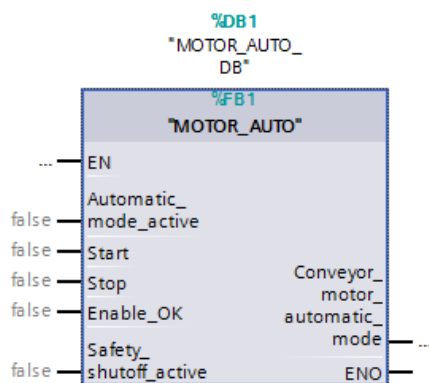
- Use drag & drop to move your "MOTOR_AUTO [FB1]" function block onto the green line in Network 1.



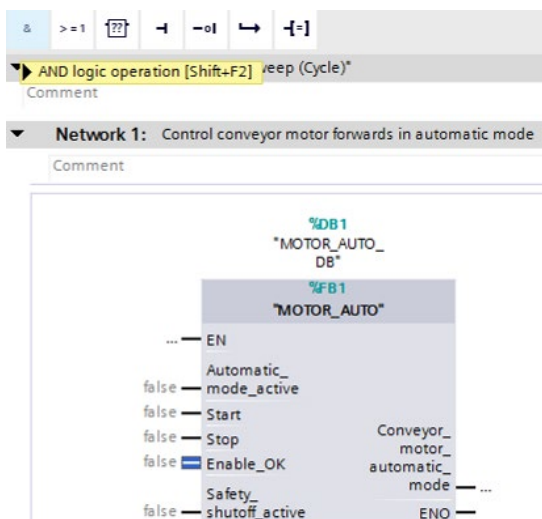
→ The instance data block for this call of FB1 is created automatically. Assign a name and apply it with OK. (→ MOTOR_AUTO_DB1 → OK)



→ A block with the interface you defined, the instance data block and connections EN and ENO are inserted in Network 1.

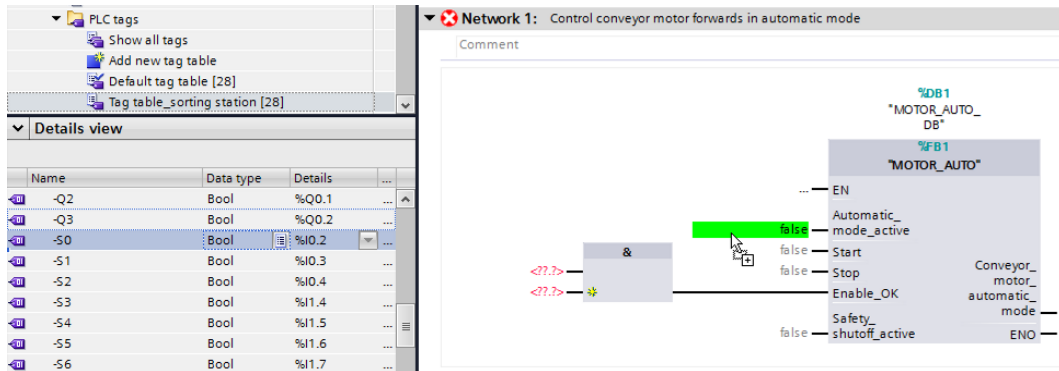


→ To insert an AND before input parameter "Enable_OK", select this input and insert the AND by clicking the  icon in your logic toolbar (→ ).



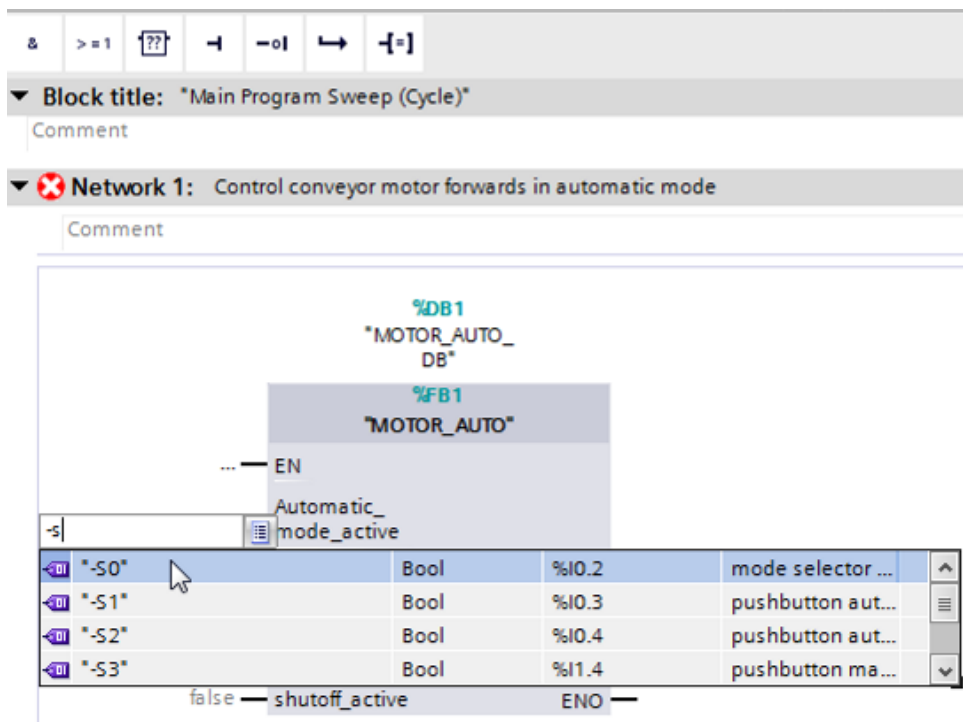
→ To connect the block to the global tags from "Tag_table_sorting_station", we have two options:

Either select the "Tag_table_sorting_station" in the project tree and use drag & drop to move the desired global tag from the Details view to the interface of FC1 (→ Tag_table_sorting_station → Details view → -S0 → Automatic_mode_active)

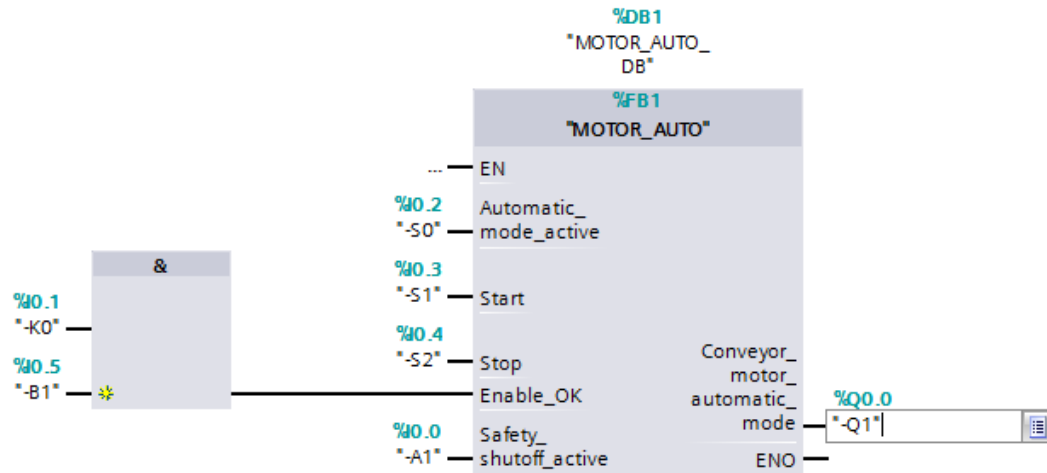


→ Or, enter the starting letters (e.g. "-S") of the desired global tag for <??.?> and select the global input tag "-S0" (%I0.2) from the displayed list

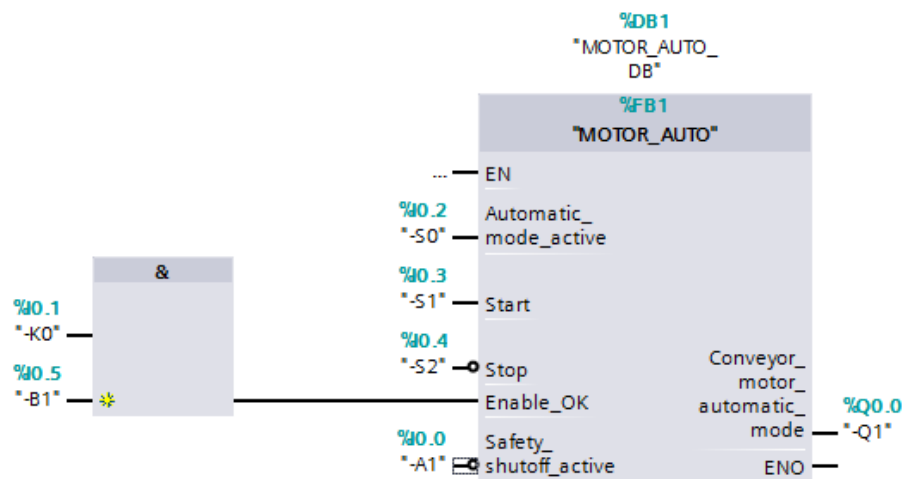
(→ Automatic_mode_active → -S → -S0).



→ Insert the other input tags "-S1", "-S2", "-K0", "-B1" and "-A1" and then insert output tag "-Q1" (%Q0.0) at output "Conveyor_motor_automatic_mode".

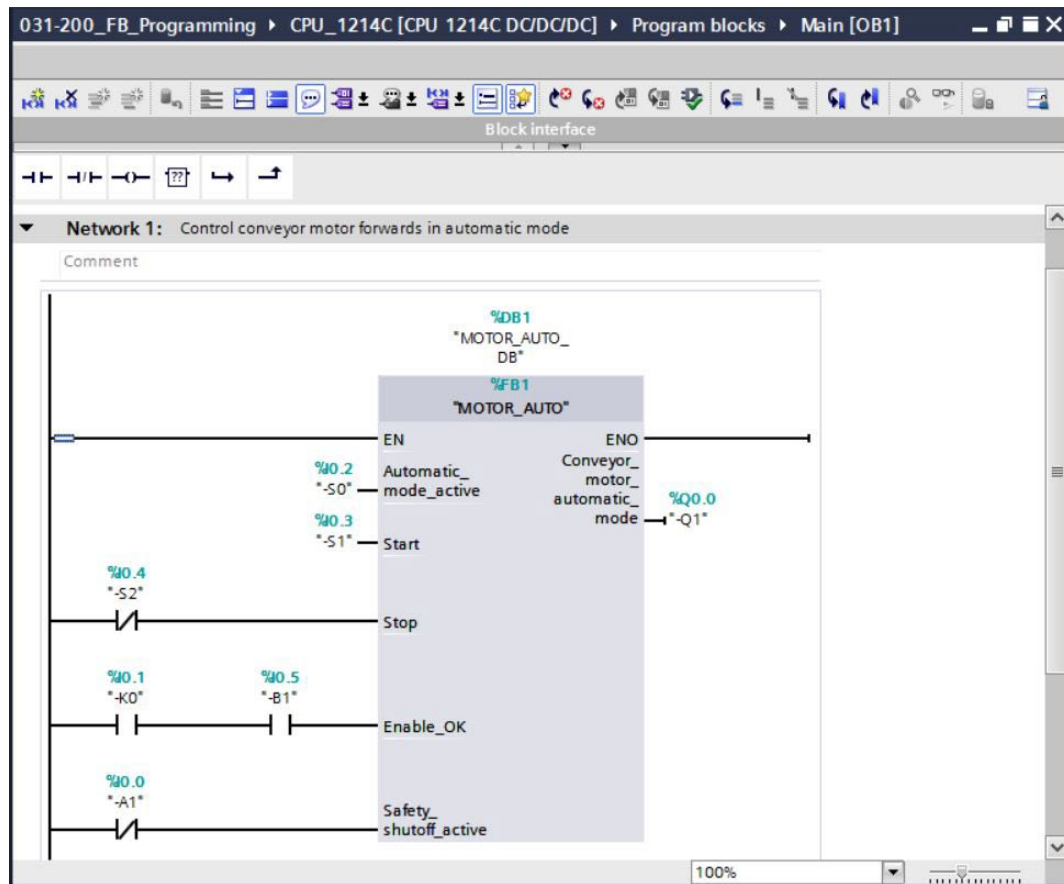


→ Negate the querying of input tags "-S2" and "-A1" by selecting them and clicking  (→ -S2 →  → -A1 → ).



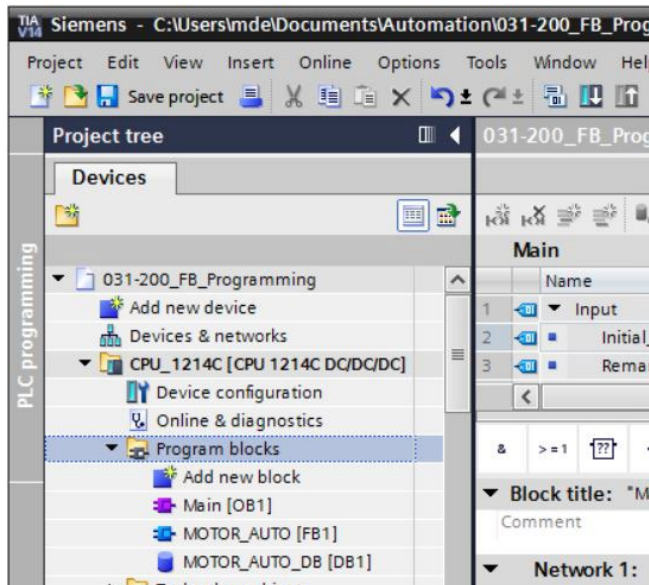
7.9 Result in the LAD (Ladder Logic) programming language

The result in the LAD (Ladder Logic) programming language has the following appearance.

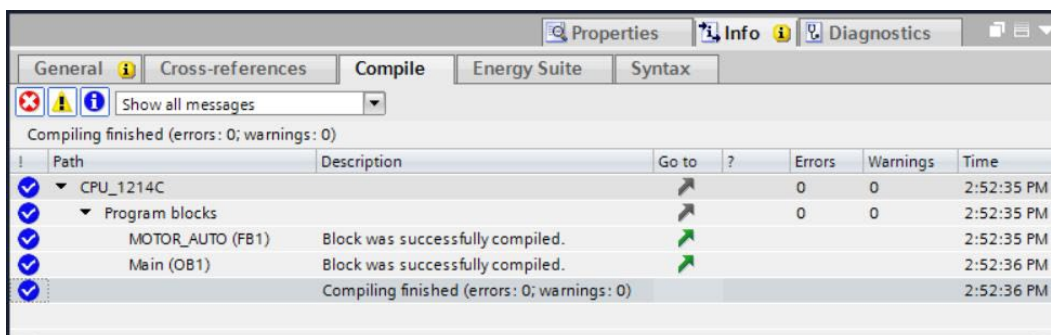


7.10 Save and compile the program

- To save your project, select the  **Save project** button in the menu. To compile all blocks, click the "Program blocks" folder and select the  icon for compiling in the menu (→  **Save project** → Program blocks → ).

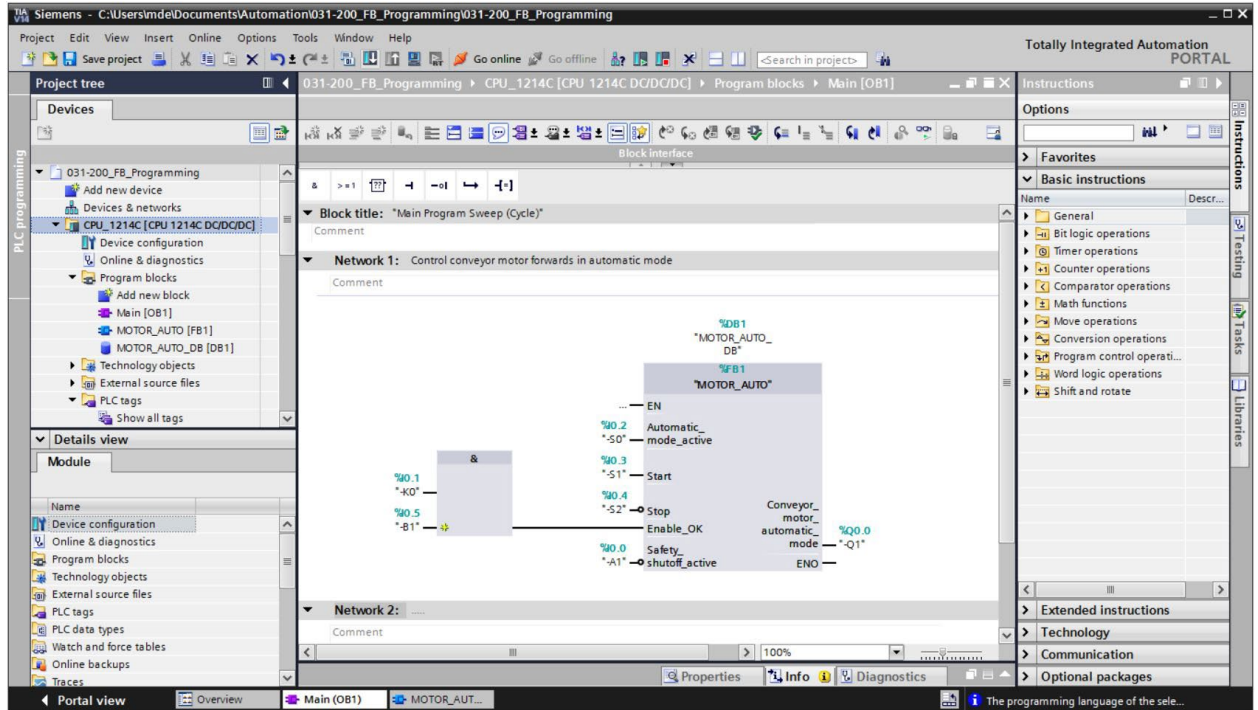


- The "Info", "Compile" area shows which blocks were successfully compiled.



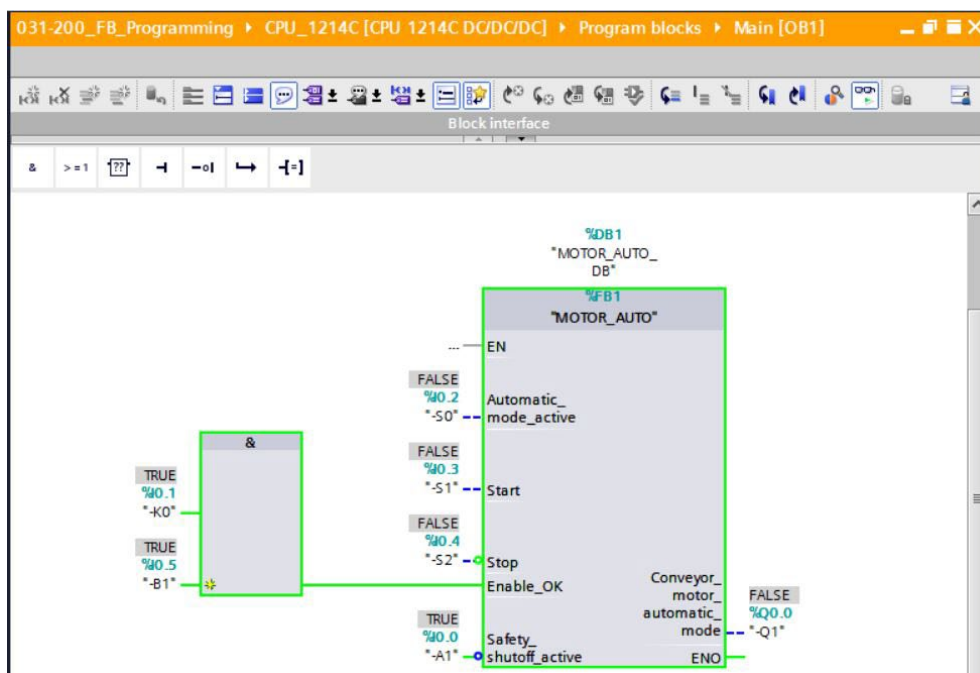
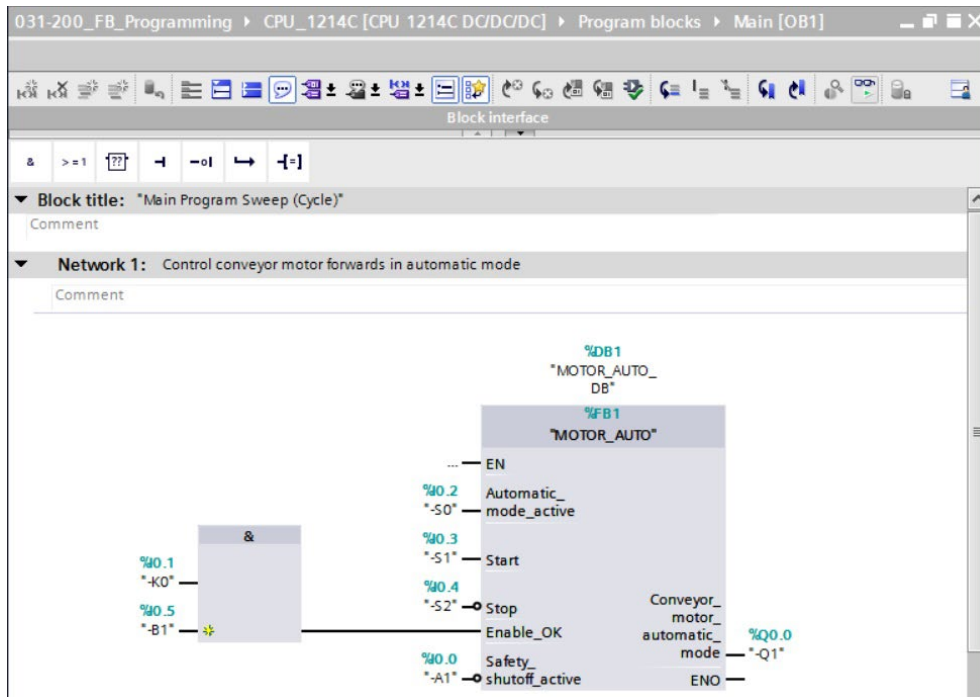
7.11 Download the program

→ After successful compilation, the complete controller with the created program, as previously described in the modules for hardware configuration, can be downloaded (→ ).



7.12 Monitor program blocks

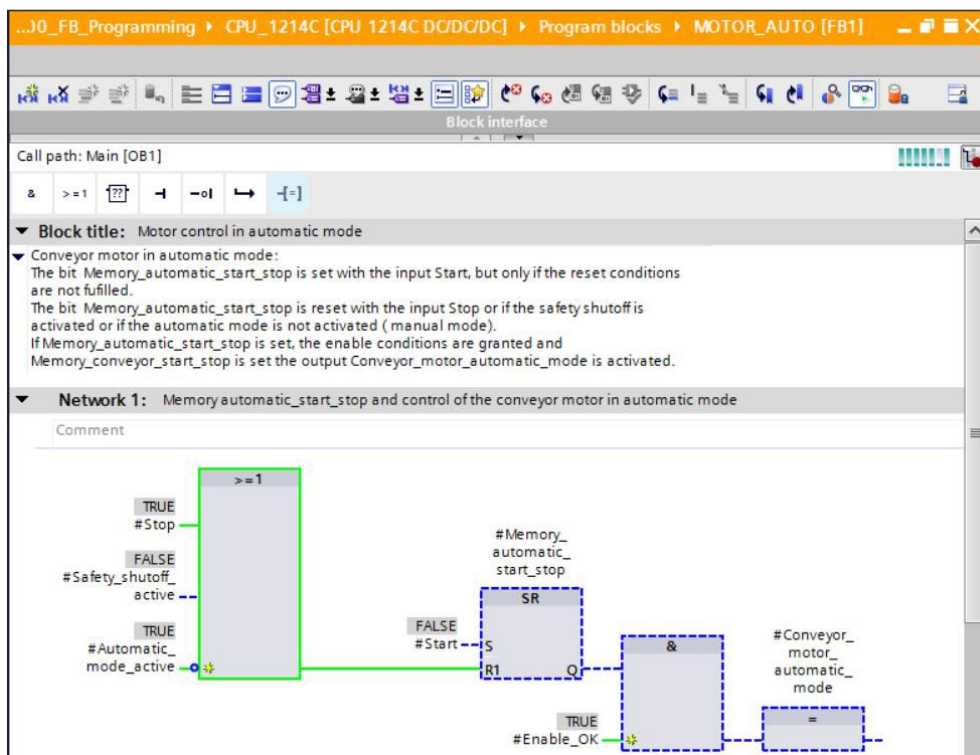
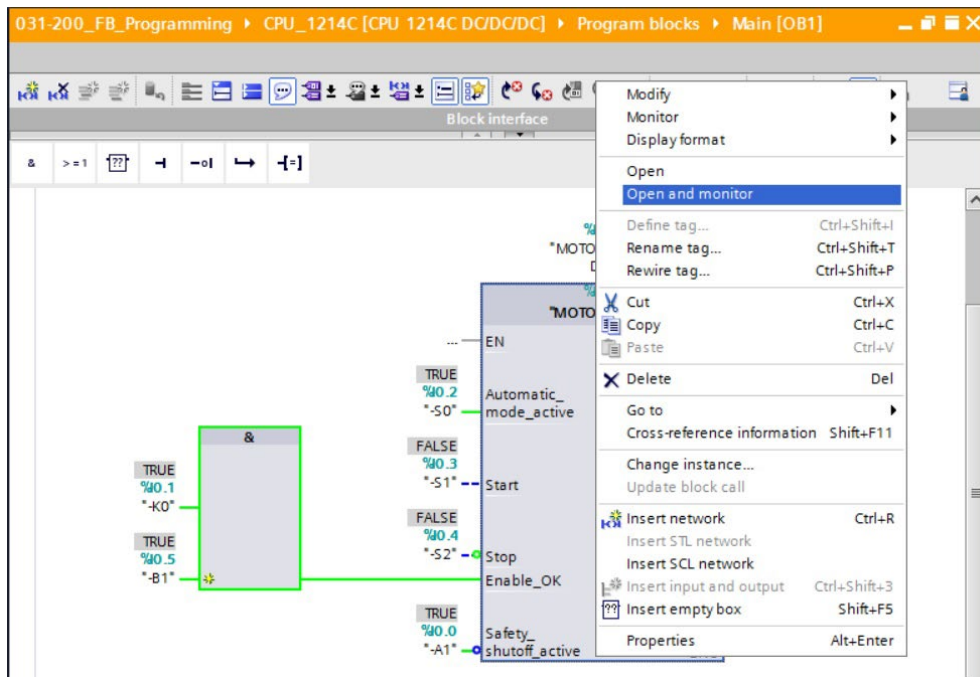
→ The desired block must be open for monitoring the downloaded program. The monitoring can now be activated/deactivated by clicking the  icon (→ Main [OB1] → ).



Note:

- The monitoring here is signal-related and controller-dependent. The signal states at the terminals are indicated with TRUE or FALSE.

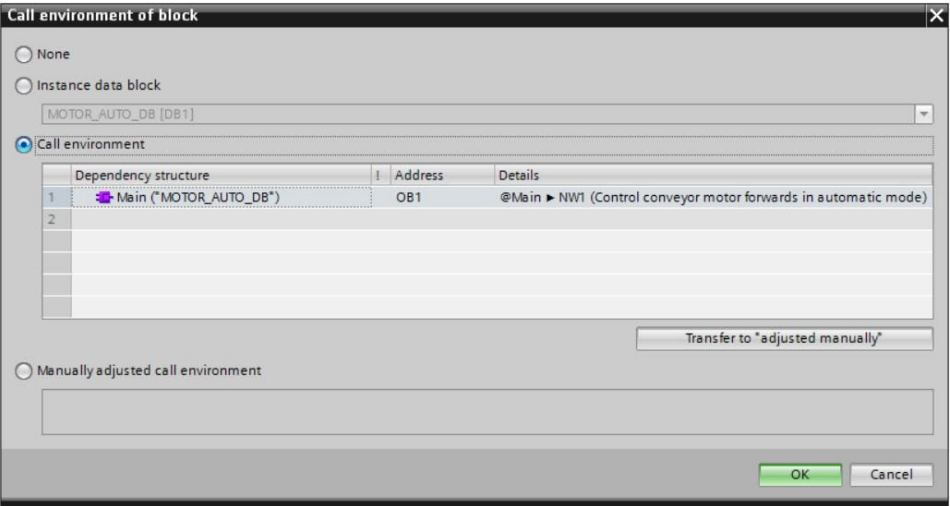
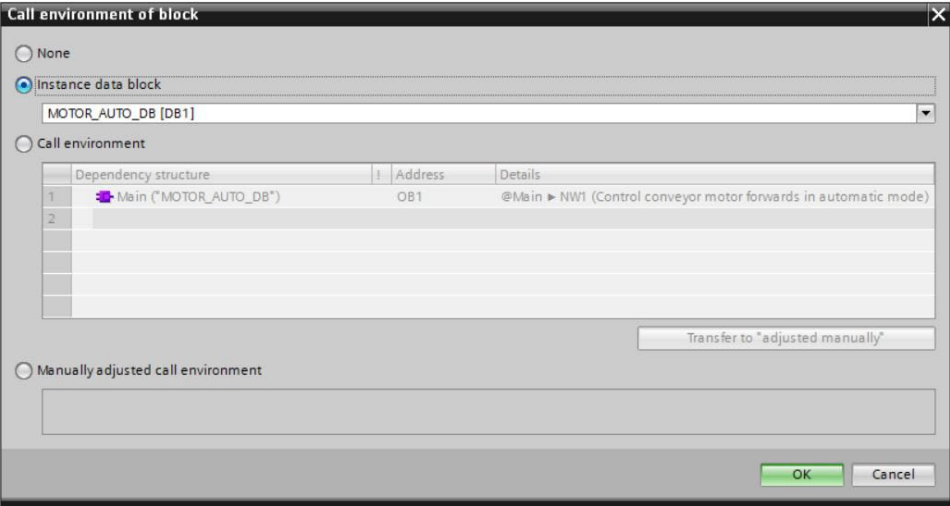
- The "MOTOR_AUTO" [FB1] function block called in the "Main [OB1]" organization block can be selected directly for "Open and monitor" after right-clicking (→ "MOTOR_AUTO" [FB1] → Open and monitor).



Note:

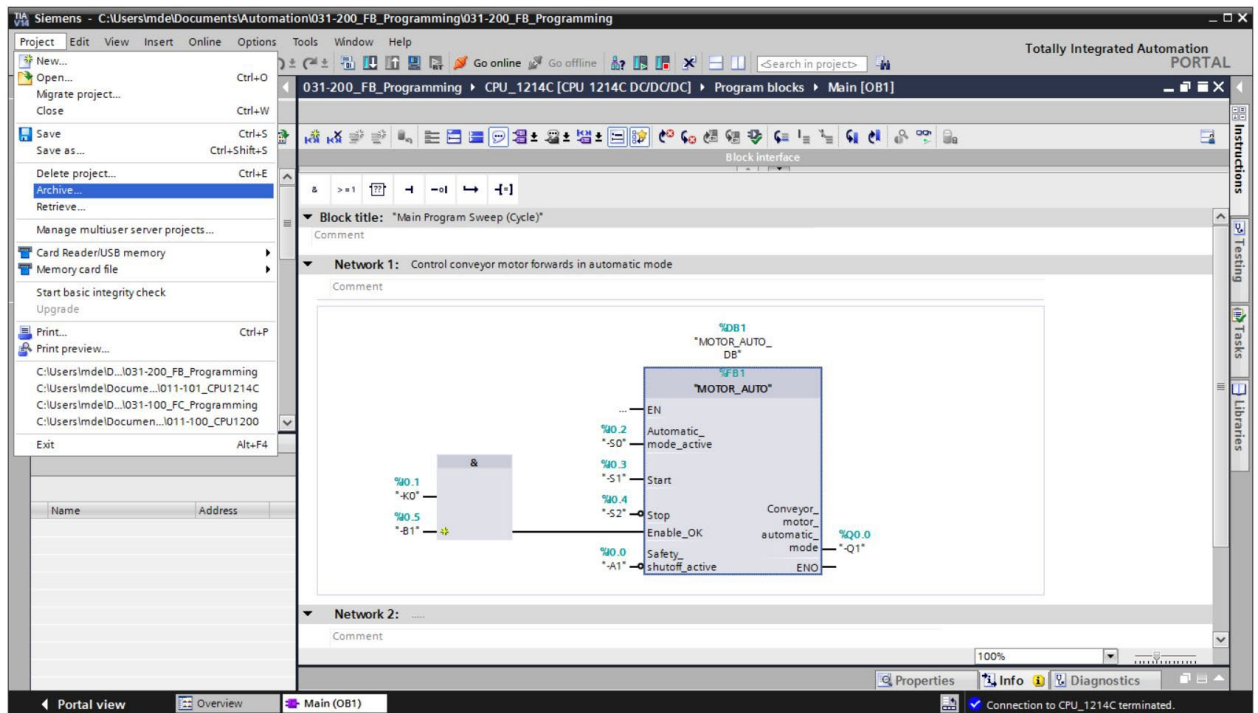
- The monitoring here is function-related and controller-independent. The actuation of sensors and the station status are shown here with TRUE or FALSE.

→ If a particular point of use of a "MOTOR_AUTO" [FB1] function block that is called multiple times is to be monitored, this can be performed using the  icon. There are two alternatives available for specifying the call environment: using the call environment or the instance data block (→  → Instance data block → MOTOR_AUTO_DB1 [DB1] → Call environment → Address: OB1 → Details: Main NW1 → OK).



7.13 Archive the project

→ As the final step, we want to archive the complete project. Select the → "Archive ..." command in the → "Project" menu. Select a folder where you want to archive your project and save it with the file type "TIA Portal project archive". (→ Project → Archive → TIA Portal project archive → 031-200_FB Programming.... → Save)



7.14 Checklist

No.	Description	Completed
1	Compiling successful and without error message	
2	Download successful and without error message	
3	Switch on station (-K0 = 1) Cylinder retracted / Feedback activated (-B1 = 1) EMERGENCY OFF (-A1 = 1) not activated AUTOMATIC mode (-S0 = 1) Pushbutton automatic stop not actuated (-S2 = 1) Briefly press the automatic start pushbutton (-S1 = 1) Conveyor motor forwards fixed speed then switches on (-Q1 = 1) switches on and stays on.	
4	Briefly press the automatic stop pushbutton (-S2 = 0) → -Q1 = 0	
5	Activate EMERGENCY OFF (-A1 = 0) → -Q1 = 0	
6	Manual mode (-S0 = 0) → -Q1 = 0	
7	Switch off station (-K0 = 0) → -Q1 = 0	
8	Cylinder not retracted (-B1 = 0) → -Q1 = 0	
9	Project successfully archived	

8 Exercise

8.1 Task – Exercise

In this exercise, an energy saving function is to be added to the MOTOR_AUTO [FB1] function block. The expanded function block will be planned, programmed and tested:

To save energy, the conveyor should only run when a part is present.

The Conveyor_motor_automatic_mode output is therefore only activated when Memory_automatic_start_stop is set, the enable conditions are met and Memory_conveyor_start_stop is set.

The Memory_conveyor_start_stop is set when Sensor_chute_occupied signals a part and is reset when Sensor_end_of_conveyor produces a negative edge or safety shutoff is active or automatic mode is not activated (manual mode).

8.2 Technology diagram

Here, you see the technology diagram for the task.

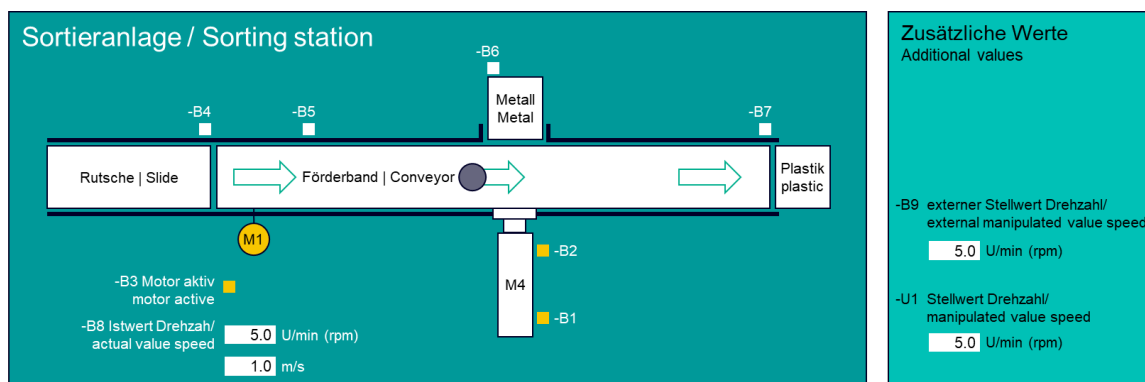


Figure 10: Technology diagram

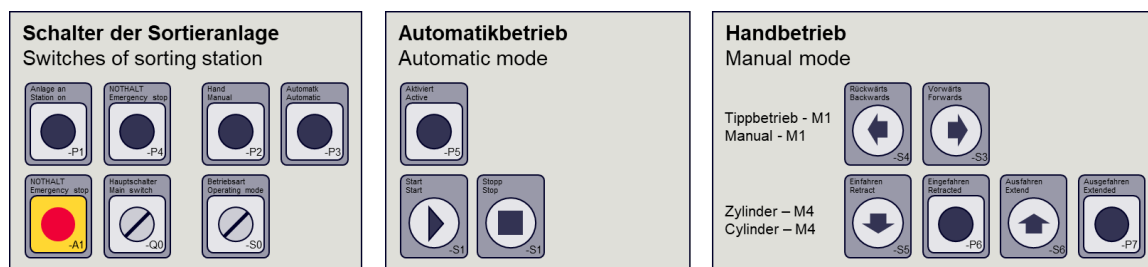


Figure 11: Control panel

8.3 Reference list

The following signals are needed as global operands for this task.

DI	Type	Identifier	Function	NC/NO
I 0.0	BOOL	-A1	Return signal emergency stop ok	NC
I 0.1	BOOL	-K0	Main switch "ON"	NO
I 0.2	BOOL	-S0	Mode selector manual (0)/ automatic (1)	Manual = 0 Auto = 1
I 0.3	BOOL	-S1	Pushbutton automatic start	NO
I 0.4	BOOL	-S2	Pushbutton automatic stop	NC
I 0.5	BOOL	-B1	Sensor cylinder M4 retracted	NO
I 1.0	BOOL	-B4	Sensor at chute occupied	NO
I 1.3	BOOL	-B7	Sensor part at end of conveyor	NO

DO	Type	Identifier	Function	
Q 0.0	BOOL	-Q1	Conveyor motor M1 forwards fixed speed	

Legend for reference list

DI	Digital Input	DO	Digital Output
AI	Analog Input	AO	Analog Output
I	Input	Q	Output
NC	Normally Closed		
NO	Normally Open		

8.4 Planning

Plan the implementation of the task on your own.

Note:

- Learn about the use of the negative edge in SIMATIC S7-1200 in the online help.

8.5 Checklist – Exercise

No.	Description	Completed
1	Compiling successful and without error message	
2	Download successful and without error message	
3	Switch on station (-K0 = 1) Cylinder retracted / Feedback activated (-B1 = 1) EMERGENCY OFF (-A1 = 1) not activated AUTOMATIC mode (-S0 = 1) Pushbutton automatic stop not actuated (-S2 = 1) Briefly press the automatic start pushbutton (-S1 = 1) Sensor at chute activated (-B4 = 1) Conveyor motor forwards fixed speed then switches on (-Q1 = 1) switches on and stays on.	
4	Sensor at end of conveyor activated (-B7 = 1) → -Q1 = 0	
5	Briefly press the automatic stop pushbutton (-S2 = 0) → -Q1 = 0	
6	Activate EMERGENCY OFF (-A1 = 0) → -Q1 = 0	
7	Manual mode (-S0 = 0) → -Q1 = 0	
8	Switch off station (-K0 = 0) → -Q1 = 0	
9	Cylinder not retracted (-B1 = 0) → -Q1 = 0	
10	Project successfully archived	

9 Additional information

More information for further practice and consolidation is available as orientation, for example: Getting Started, videos, tutorials, apps, manuals, programming guidelines and trial software/firmware, at the following link: siemens.com/sce/quicklinks

Preview "SCE Quick Links"

Topics

Siemens SCE siemens.com/sce	Digital Enterprise siemens.com/digital-enterprise	Discover SIMIT Simulation support.industry.siemens.com/cs/ww/en/view/101298066
SCE SIOS Learning Packages siemens.com/sce/lo	Totally Integrated Automation (TIA) siemens.com/tia	SIMIT Simulation Software support.industry.siemens.com/cs/ww/en/ps/17120ld
SCE Learning & Training Documents siemens.com/sce/documents	TIA Portal siemens.com/tia-portal	SIMATIC Technical Documentation siemens.com/simatic-docu
SCE Trainer Packages siemens.com/sce/tp	TIA Selection Tool siemens.com/tia/tia-selection-tool	Support Inquiries ?
SCE Trial Software/Firmware siemens.com/sce/support	SIMATIC Controllers siemens.com/controller	MindSphere Academia Inquiries ? siemens.mindsphere.io/en/product-description-overview/mindsphere-academia
SCE Contact Partners siemens.com/sce/contact	Webinars siemens.com/sce/webinars	PLM Academic Inquiries ? plm.automation.siemens.com/global/de/our-story/partners/academic/educator/
SCE WorldSkills siemens.com/worldskills	Use Cases siemens.com/sce/usecases	SiePortal – Siemens Industry Mall & Online Support siemens.com/sieportal

Quick Links

1. Digital learning modules (theory documents, learning videos, interactive call-to-action learning videos as well as projects)

- 1.1 [Basic Course TIA Factory Automation with SIMIT](#)
- 1.2 [Basic Course TIA Factory Automation with SIMIT and NX MCD](#)
- 1.3 [Advanced Course Digital Twin](#)

2. Learning & Training Documents (Step-by-step instructions via Word or PDF document incl. projects and check lists)

- 2.1 [TIA Portal Modules](#)
- 2.2 [PCS 7 Modules](#)
- 2.3 [CNC Modules](#)
- 2.4 [LOGO! Modules](#)

More information

Siemens Automation Cooperates with Education

siemens.com/sce

SCE Learning/Training Documents

siemens.com/sce/module

SCE Trainer Packages

siemens.com/sce/tp

TIA Portal & SIMIT Software for Educators/Students

siemens.com/sce/support

SCE Contact Partners

siemens.com/sce/contact

PLM Contact Partners

siemens.com/plm/gaf

PLM Academic Partner Program

plm.automation.siemens.com/global/en/our-story/partners/academic/

Discover SIMIT Simulation

support.industry.siemens.com/cs/ww/en/view/101298066

SIMIT Simulation Software

support.industry.siemens.com/cs/de/en/ps/17120/dl

Digital Enterprise

siemens.com/digital-enterprise

Totally Integrated Automation (TIA)

siemens.com/tia

TIA Portal

siemens.com/tia-portal

TIA Selection Tool

siemens.com/tia/tia-selection-tool

SIMATIC Controllers

siemens.com/controller

SIMATIC Technical Documentation

siemens.com/simatic-docu

Industry Online Support

support.industry.siemens.com

Industry Mall catalog and online ordering system

mall.industry.siemens.com

Published by

Siemens AG

Digital Industries

Factory Automation

P.O. Box 48 48

90026 Nuremberg, Germany

Errors excepted and subject to change without prior notice. The information provided in this document contains merely general descriptions or characteristics of performance which in case of actual use do not always apply as described or which may change as a result of further development of the products. An obligation to provide the respective characteristics shall only exist if expressly agreed in the terms of contract.

siemens.com/sce